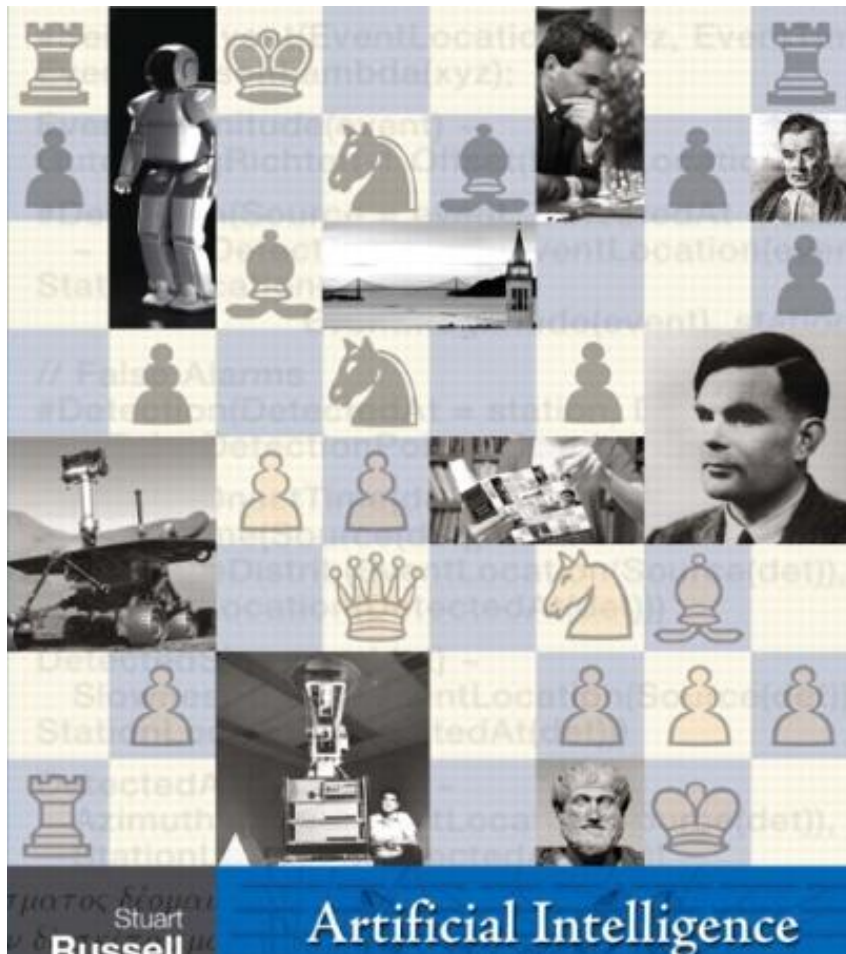


Lecture 8: Search Optimization- Part I



CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

**Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)**

Disclosure

- We will be using following article in our Search Optimization lectures

Ab Wahab MN, Nefti-Meziani S, Atyabi A (2015) A Comprehensive Review of Swarm Optimization Algorithms. PLoS ONE 10(5): e0122827.
doi:10.1371/journal.pone.0122827

Swarm Intelligence (SI)

- Bonabeau defined SI as “*The emergent collective intelligence of groups of simple agents*” [1].
- SI is the collective intelligent behavior of self-organized and decentralized systems, e.g., artificial groups of simple agents.
- Two fundamental concepts that are considered as necessary properties of SI are self-organization and division of labor.

Swarm Intelligence (SI)

- Self-organization is defined as the capability of a system to evolve its agents or components into a suitable form without any external help.
- Self-organization relies on four fundamental properties of 1) positive feedback, 2) negative feedback, 3) fluctuations and 4) multiple interactions.
 - Positive and negative feedbacks are useful for amplification and stabilization respectively.
 - Fluctuations meanwhile are useful for randomness.
 - Multiple interactions occur when the swarms share information among themselves within their searching area.
- Division of labor is defined as the simultaneous execution of various simple and feasible tasks by individuals. This division allows the swarm to tackle complex problems that require individuals to work together [1].

Swarm Intelligence (SI)

- Several algorithms exist that are inspired from principles of SI including
 - **Genetic Algorithms (GA)**,
 - **Differential Evolution (DE)**,
 - **Particle Swarm Optimization (PSO)**,
 - **Ant Colony Optimization (ACO)**,
 - Artificial Bee Colony (ABC),
 - Glowworm Swarm Optimization (GSO),
 - Cuckoo Search Algorithm (CSA)
 - Evolution Strategy (ES),
 - Evolutionary Programming (EP),
 - Firefly Algorithm (FA),
 - Bat Algorithm (BA)
 - Grey Wolf Optimizer (GWO)
 - And others ...

Genetic Algorithm (GA)

- The Genetic Algorithm (GA) introduced by John Holland in 1975.
- GA is a search optimization algorithm based on the mechanics of the natural selection process.
- The basic concept of this algorithm is to mimic the concept of the 'survival of the fittest';
- It simulates the processes observed in a natural system where the strong tends to adapt and survive while the weak tends to perish.

Genetic Algorithm (GA)

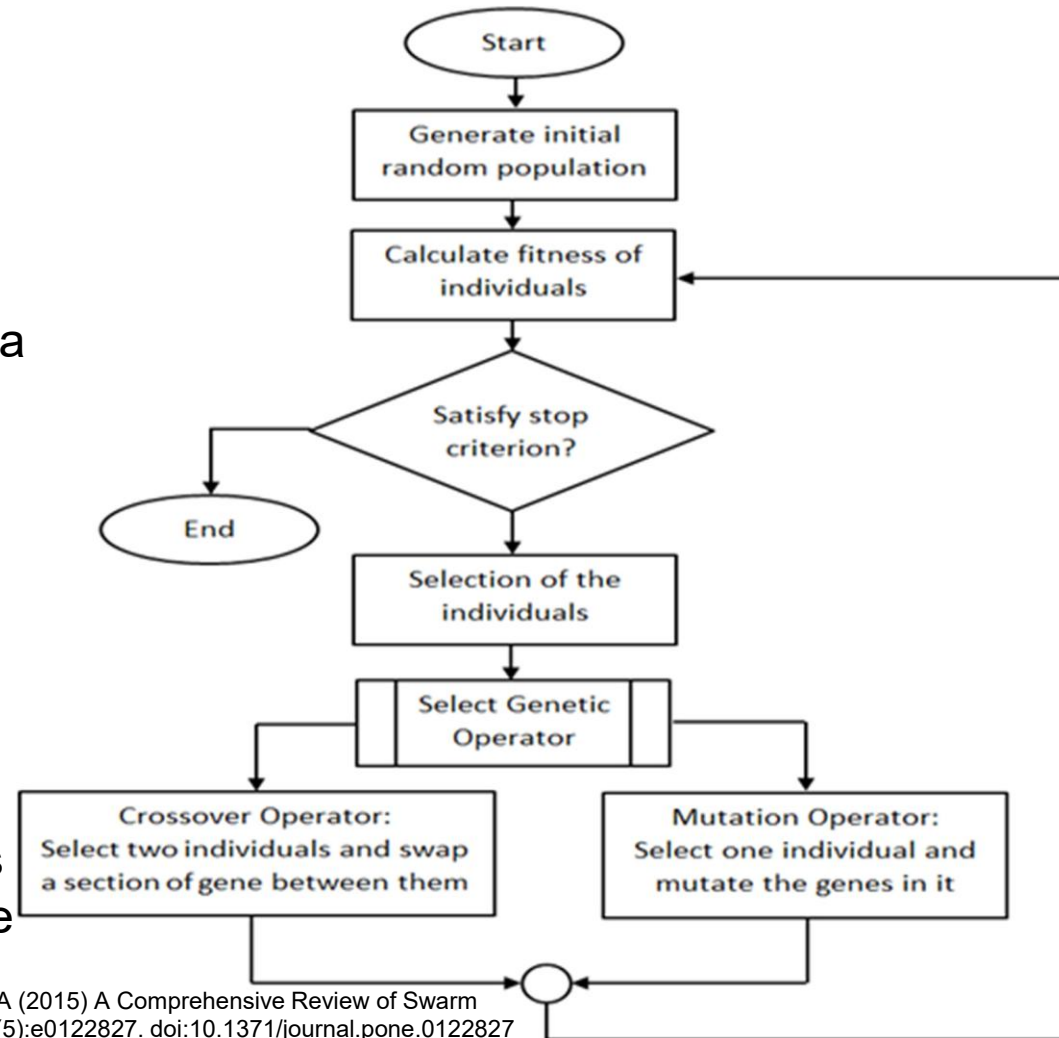
- GA is a population-based approach in which members of the population are ranked based on their solutions' fitness.
- In GA, a new population is formed using specific genetic operators such as crossover, reproduction, and mutation [4–7].
- Population can be represented in a set of strings (referred to as chromosomes). In each generation, a new chromosome (a member of the population) is created using information originated from the fittest chromosomes of the previous population [4–6].

Genetic Algorithm (GA)

- GA generates an initial population of feasible solutions and recombines them in a way to guide their search towards more promising areas of the search space.
- Each of these feasible solutions is encoded as chromosome, also referred to as genotype, and each of these chromosomes will get a measure of fitness through a fitness function (evaluation or objective function).
- The value of fitness function of a chromosome determines its capability to endure and produce offspring. The high fitness value indicates the better solution for maximization and the low fitness value shows the better solution for minimization problems.

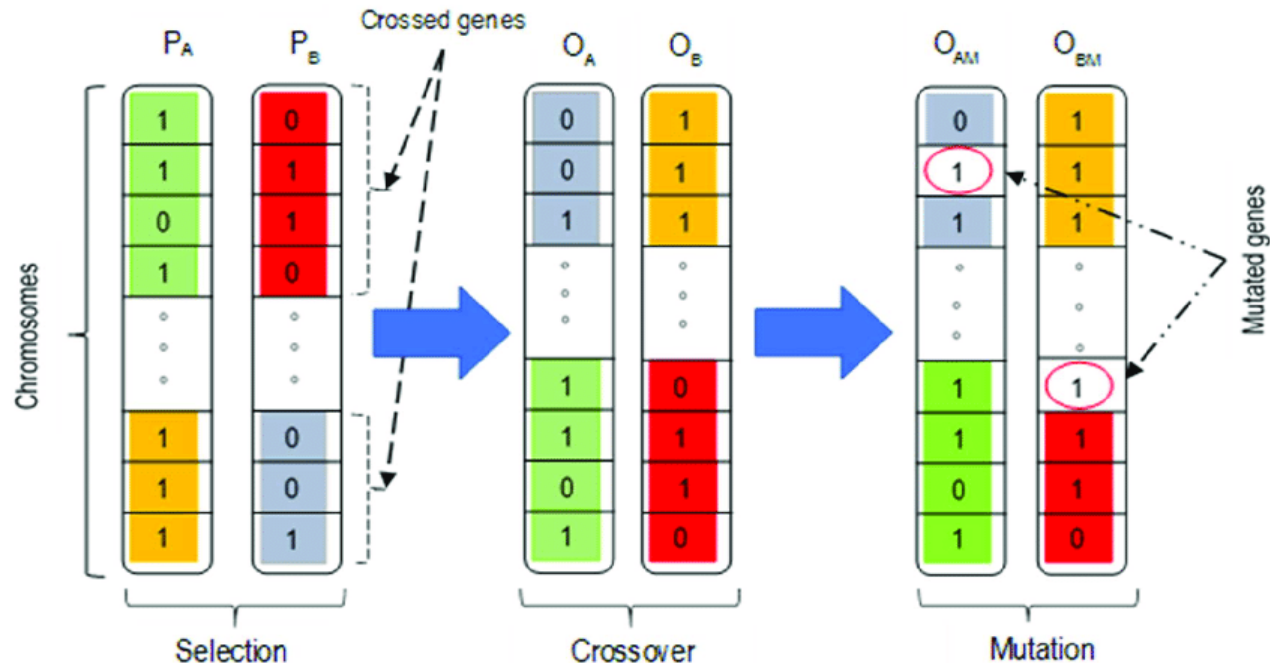
Genetic Algorithm (GA)

- A basic GA has five main components: a random number generator, a fitness evaluation unit, a reproduction process, a crossover process, and a mutation operation.
- Reproduction selects the fittest candidates of the population, while crossover is the procedure of combining the fittest chromosomes and passing superior genes to the next generation, and mutation alters some of the genes in a chromosome [4–7].



Ab Wahab MN, Nefti-Meziani S, Atyabi A (2015) A Comprehensive Review of Swarm Optimization Algorithms. PLoS ONE 10(5):e0122827. doi:10.1371/journal.pone.0122827

Genetic Algorithm (GA)



- Chinedu I. Ossai, Prognosis and Remaining Useful Life Estimation of Lithium-Ion Battery with Optimal Multi-Level Particle Filter and Genetic Algorithm, Batteries 2018, 4, 15; doi:10.3390/batteries4020015

Genetic Algorithm (GA)

- GA is advantageous in terms of requiring limited parameter settings and initializing itself from possible solutions rather than a single solution.
- One of the main drawbacks of GA is the lack of fast convergence towards the optimal values since the crossover and mutation process are random [6, 7].
- The applications of GA are wide ranging from scheduling [8, 9], machine learning [10], robotics [11, 12], signal processing [13], business [14], mathematics [15], manufacturing [16, 17], routing [18], and many more.

Ways to modify GA

Crossover & Mutation

- In crossover, instead of selecting one crossover point, De Jong et al. (1992) and Üçoluk (2002) have introduced N-point crossover and segmented crossover which selects several points for crossover [19, 20]. The difference between them is N-point crossover is choosing several breaking points randomly, while in segmented crossover, only two breaking points are utilized.
- Several studies have given different methods for mutation. By default, each gene in a chromosome is assigned with probability, p_m , and mutated depending on that probability. This mutation is known as uniform mutation. The other approaches for mutation are bitwise inversion where the whole gene in a chromosome is mutated using a random mutation [19].

Ways to modify GA

Crossover & Mutation

- Adaptive genetic algorithms have been introduced in order to allow the use of precise parameters in setting the population size, the crossing over probability, and the mutation probability. All of these parameters are dynamic and changing over the iterations.
- Reinitialization: Raja and Bhaskaran [22] have utilized initialization twice where the first initialization is used to identify the promising area. After the first initialization, all chromosomes are ranked and the best chromosomes are selected. After that, GA is initialized again within the area where the best chromosomes have been identified.

Particle Swarm Optimization

- PSO is an Evolutionary Algorithm inspired from animal social behaviors. (Kennedy, 1995, Ribeiro and Schlansker, 2005; Chang et al., 2004; Pugh and Martinoli, 2006; Sousa et al., 2003; Nomura, 2007)
- PSO outperformed other Evolutionary Algorithms such as GA in some problems (Vesterstrom and Riget, 2002; Ratnaweera et al., 2004; Pasupuleti and Battiti, 2006).
- PSO is an optimization technique which models a set of potential problem solutions as a swarm of particles moving about in a virtual search space. (Kennedy, 1995)

Particle Swarm Optimization

- The method was inspired by the movement of flocking birds and their interactions with their neighbors in the group. (Kennedy, 1995)
- PSO achieves optimization using three primary principles: (Pugh, Martinoli, 2007)
 - Evaluation, where quantitative fitness can be determined for some particle location;
 - Comparison, where the best performer out of multiple particles can be selected;
 - Imitation, where the qualities of better particles are mimicked by others.

Particle Swarm Optimization

- Velocity: (1)

$$\vec{v}_{i,j}(t) = w \vec{v}_{i,j}(t-1) + C_{i,j} + S_{i,j}$$

$$C_{i,j} = c_1 r_{1,j} \cdot (\vec{p}_{i,j}(t-1) - \vec{x}_{i,j}(t-1))$$

$$S_{i,j} = c_2 r_{2,j} \cdot (\vec{g}_{i,j}(t-1) - \vec{x}_{i,j}(t-1))$$

- Maximum Velocity (2)

$$V_{i,j} = \begin{cases} V_{min} & \text{if } V_{i,j} \leq V_{min} \\ V_{max} & \text{if } V_{i,j} \geq V_{max} \\ V_{i,j} & \text{otherwise} \end{cases}$$

- Inertia Weight (2-1) $w = (w_1 - w_2) \times \frac{(maxiter - t)}{maxiter} + w_2$

Particle Swarm Optimization

- Solutions (positions): (3)

$$\vec{x}_{i,j}(t+1) = \vec{x}_{i,j}(t) + \vec{v}_{i,j}(t+1)$$

- Personal best and Global Best (4)

$$P_i(t) = \begin{cases} P_i(t-1) & \text{if } f(x_i(t)) \geq f(P_i(t-1)) \\ x_i(t) & \text{otherwise} \end{cases}$$

$$g(t) = \operatorname{argmin}\{f(P_1(t)), f(P_2(t)), \dots, f(P_s(t))\}$$

Problems and Solutions in Basic PSO

- **Problems**

- Premature convergence
- Parameter control
- Diversity (exploration, exploitation)
- Dynamic domains
- Real Time domains

- **Solutions**

- Improvement on neighborhood topology, velocity equation, global best and personal best.
- Niching PSO, Mutation, Parallelism, Reinitialization, Clearing memory, Using Sub-Swarms

(Brits, Engelbrecht, and Van Den Bergh, 2002,2003; Yoshida, et al.,2001; Stacey, Jancic and Grundy,2003;Chang, et al., 2005; Vestestrom, Riget, 2002; Qin et al., 2004; Pasupuleti and Battiti, 2006; Ratnaweera et al., 2004;Peram et al., 2003; Parsopoulos and Vrahatis, 2002)

Various Modeling of PSO

- Two main implementation of PSO are known as i) Macroscopic and ii) Microscopic models.
- Microscopic Modeling: refer to implementations in which each agent represent a swarm by itself and particles of that swarm, represent the possible acts for that individual agent as the next move.
- Macroscopic Modeling: refer to implementations in which each agent represent a particle of the swarm therefore, the whole team represent the swarm.

(Agassounon et al., 2004, Chang et al., 2005, Kazadi et al., 2002, Lerman et al., 2001, Martinoli et al., 2007, Pugh and Martinoli 2007a, Li et al., 2004)

Modifying PSO formulation

$$\vec{V}(t+1) = \text{fittest} \left\{ \begin{array}{llll} \phi_2(g(t) - x(t)) & & & : 1 \\ \phi_1(p(t) - x(t)) & & & : 2 \\ w \times V(t) & & & : 3 \\ 1 + 2 & HPSO & & : 4 \\ 1 + 3 & GPSO & & : 5 \\ 2 + 3 & GCPSO & & : 6 \\ 1 + 2 + 3 & Basic - PSO & & : 7 \end{array} \right.$$

$$\text{where } \phi_1 = c_1 \times \text{rand}()$$

$$\phi_2 = c_2 \times \text{rand}()$$

Performance Comparison: Benchmark Problems

- There are many optimization techniques claiming superiority over other approaches.
- In order to determine the most reliable algorithms, benchmark functions can be used as indicator to prove their effectiveness.
- Several benchmark functions with different properties have been used to evaluate the feasibility of optimization algorithms;

Performance Comparison: Benchmark Problems

- There are 3 properties that are considered in benchmark functions called **multimodality**, **separability**, and **regularity**.
- A function is multimodal if it has two or more local optima.
- A function of p variables is separable if it can be rewritten as a sum of p functions of just one variable. The separability is closely related to the concept of epistasis or interrelation among the variables of the function. In the field of evolutionary computation, the epistasis measures how much the contribution of a gene to the fitness of the individual depends on the values of other genes.

Performance Comparison: Benchmark Problems

- Non separable functions are more difficult to optimize as the accurate search direction depends on two or more genes. On the other hand, separable functions can be optimized for each variable in turn. The problem is even more difficult if the function is also multimodal. The search process must be able to avoid the regions around local minima in order to approximate, as far as possible, the global optimum. The most complex case appears when the local optima are randomly distributed in the search space.
- The dimensionality of the search space is another important factor in the complexity of the problem. A study of the dimensionality problem and its features was carried out by Friedman [Fri94].

Performance Comparison: Parameter settings

- **GA:**

- Max iteration numbers is set to 100
- Population sizes is set to 100
- Single point crossover type is used with 1 crossover probability.
- Mutation probability is set to 0.01
- the best two solutions are selected for elitism.

- **PSO:**

- Max iteration numbers is set to 100
- Population sizes is set to 100
- Inertia weight value used is 0.728994
- acceleration coefficients (c_1, c_2) for cognitive and social are set to 0.5 and 2.5 respectively.

Performance Comparison: Benchmark Problems

Table 2. Benchmark Functions Selected for Comparison.

No	Function	Formula	Value	Dim	Range	Properties
1	Sumsquare	$f(x) = \sum_{i=1}^n x_i^2$	0	30	[-5.12, 5.12]	Unimodal, Separable
2	Sphere	$f(x) = \sum_{i=1}^n x_i^2$	0	30	[-100, 100]	Unimodal, Separable
3	Beale	$f(x) = (1.5 - x_1 + x_1 x_2)^2 + (2.25 - x_1 + x_1 x_2^2)^2 + (2.625 - x_1 + x_1 x_2^3)^2$	0	2	[-4.5, 4.5]	Unimodal, Inseparable
4	Convex	$f(x) = 100(x_1^2 - x_2)^2 + (x_1 - 1)^2 + (x_2 - 1)^2 + 90(x_2^2 - x_1)^2 + 10.1((x_1 - 1)^2 + (x_2 - 1)^2) + 19.8(x_2 - 1)(x_1 - 1)$	0	4	[-10, -10]	Unimodal, Inseparable
5	Decon-Price	$f(x) = -(x_1 - 1)^2 + \sum_{i=2}^n f(2x_i^2 - x_i - 1)^2$	0	24	[-5, 5]	Unimodal, Inseparable
6	Easom	$f(x) = -\cos(x_1)\cos(x_2) \exp(-(x_3 - \pi)^2 - (x_4 - \pi)^2)$	0	30	[-80, 80]	Unimodal, Inseparable
7	Matyas	$f(x) = 0.26(x_1^2 + x_2^2) - 0.48x_1x_2$	0	2	[-10, 10]	Unimodal, Inseparable
8	Powell	$f(x) = \sum_{i=1}^{n-1} (x_{i+1} - 10x_{i-1})^2 + 5(x_{i+1} + x_i)^2 + (x_{i+1} - x_i)^4 + 1.0(x_{i+1} + x_i)^4$	0	2	[-100, 100]	Unimodal, Inseparable
9	Rosenbrock	$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	-1	2	[-100, 100]	Unimodal, Inseparable
10	Schwefel	$f(x) = \sum_{i=1}^n -x_i \sin(\sqrt{ x_i })$	0	30	[-500, 500]	Unimodal, Inseparable
11	Trid6	$f(x) = \sum_{i=1}^n (x_i - 1)^2 - \sum_{i=1}^n x_i x_{i+1}$	-50	6	$[-D^2, D^2]$	Unimodal, Inseparable
12	Zakharov	$f(x) = \sum_{i=1}^n x_i + \left(\sum_{i=1}^n 0.5x_i\right)^2 + \left(\sum_{i=1}^n 0.5x_i\right)^4$	0	10	[-5, 10]	Unimodal, Inseparable
13	Bohachevsky1	$f(x) = x_1^2 + 2x_2^2 - 0.3\cos(3\pi x_1) - 0.4\cos(4\pi x_2) + 0.7$	0	2	[-100, 100]	Multimodal, Separable
14	Booth	$f(x) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$	0	2	[-10, 10]	Multimodal, Separable
15	Brannin	$f(x) = (x_2 - \frac{1}{4}x_1^2 + \frac{1}{2}x_1^2 - 6)^2 + 10(1 - \frac{1}{10})\cos x_1 + 10$	0.398	2	[-5, 10] x [0, 10]	Multimodal, Separable
16	Michalewicz5	$f(x) = -\sum_{i=1}^n \sin(x_i) \left(\sin(2x_i^2/\pi)\right)^{20}$	-4.688	5	[0, π]	Multimodal, Separable
17	Rastrigin	$f(x) = \sum_{i=1}^n x_i^2 - 10\cos(2\pi x_i) + 10$	0	30	[-5.12, 5.12]	Multimodal, Separable
18	Shubert	$f(x) = \left(\sum_{i=1}^n \cos((i+1)x_i + i)\right) \left(\sum_{i=1}^n i \cos((i+1)x_i + i)\right)$	-186.73	2	[-10, 10]	Multimodal, Separable
19	Ackley	$f(x) = -20 \exp\left(-0.2\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}\right) - \exp\left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)\right) + 20 + e$	0	30	[-32, 32]	Multimodal, Inseparable
20	Bohachevsky2	$f(x) = x_1^2 + 2x_2^2 - 0.3\cos(3\pi x_1)\cos(4\pi x_2) + 0.3$	0	2	[-100, 100]	Multimodal, Inseparable
21	Bohachevsky3	$f(x) = x_1^2 + 2x_2^2 - 0.3\cos(3\pi x_1 + 4\pi x_2) + 0.3$	0	2	[-100, 100]	Multimodal, Inseparable
22	Bukin6	$f(x) = 100\sqrt{ x_2 - 0.01x_1^2 } + 0.01 x_1 + 10 $	0	2	$x_1 \in [-15, -5], x_2 \in [-3, 3]$	Multimodal, Inseparable
23	Drop-Wave	$f(x) = -\frac{1+\cos(\frac{\pi\sqrt{x_1^2+x_2^2}}{2})}{\sin(\frac{\pi x_1^2}{2})+1}$	-1	2	[-5.12, 5.12]	Multimodal, Inseparable
24	Eggholder	$f(x) = -(x_1 + 47)\sin\left(\sqrt{ x_1 + \frac{x_2}{47} + 47 }\right) - x_1 \sin(\sqrt{ x_1 + (x_2 + 47) })$	-959.6407	2	[-5.12, 5.12]	Multimodal, Inseparable
25	Goldstein-Price	$f(x) = [1 + (x_1 + x_2 + 1)^4(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)](30 + (2x_1 - 3x_2)^2(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2))$	0	2	[-10, 10]	Multimodal, Inseparable
26	Griewank	$f(x) = \frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos\frac{x_i}{\sqrt{i}} + 1$	0	30	[-600, 600]	Multimodal, Inseparable
27	McCormick	$f(x) = \sin(x_1 + x_2) + (x_1 + x_2)^2 - 1.5x_1 + x_2^2 + 1$	-1.9133	2	$x_1 \in [-1.5, 4], x_2 \in [-3, 4]$	Multimodal, Inseparable

(Continued)

Ab Wahab MN, Nefti-Meziani S, Atiyabi A (2015) A Comprehensive Review of Swarm Optimization Algorithms. PLoS ONE 10(5):e0122827. doi:10.1371/journal.pone.0122827

Performance Comparison

Table 3. Benchmark Functions Comparison of mean error (Mean $\pm$ SD) and time (Second) on Several Optimization Techniques.

Function	GA	PSO
Sphere (Separable)	6.4415E+03	1.0454E+05
	$\pm 1.6876E+03$	$\pm 7.1998E+04$
	(4.3531s)	(2.8906s)
Sumsquare (Separable)	1.7376E+01	3.7357E+00
	$\pm 3.7449E-15$	$\pm 1.8203E-01$
	(3.8938s)	(3.1422s)
Beale (Inseparable)	7.0313E-01	0.0000E+00
	$\pm 0.0000E+00$	$\pm 0.0000E+00$
	(3.1078s)	(1.9094s)
Colville (Inseparable)	0.0000E+00	0.0000E+00
	$\pm 0.0000E+00$	$\pm 0.0000E+00$
	(2.6875s)	(1.8922s)
Dixon-Price (Inseparable)	1.1029E+05	4.9633E+06
	$\pm 3.4184E+04$	$\pm 3.4317E+06$
	(4.1063s)	(3.1000s)
Easom (Inseparable)	-1.0000E+00	-1.0000E+00
	$\pm 0.0000E+00$	$\pm 0.0000E+00$
	(6.0281s)	(1.8875s)
Matyas (Inseparable)	0.0000E+00	0.0000E+00
	$\pm 0.0000E+00$	$\pm 0.0000E+00$
	(2.7453s)	(2.0250s)
Powell (Inseparable)	4.2230E+02	1.0689E+04
	$\pm 1.2382E+02$	$\pm 3.7167E+03$
	(3.9266s)	(2.6859s)
Rosenbrock (Inseparable)	1.2493E+07	7.0289E+08
	$\pm 8.6725E+06$	$\pm 4.8937E+08$
	(4.0797s)	(2.9797s)
Schwefel (Inseparable)	5.2808E+03	7.0202E+03
	$\pm 6.2830E+02$	$\pm 1.2171E+02$
	(4.4391s)	(2.8094s)
Trid6 (Inseparable)	-2.5000E+01	-5.0000E+01
	$\pm 1.2293E+01$	$\pm 0.0000E+00$
	(3.1516s)	(2.0859s)
Zakharov (Inseparable)	2.9550E+01	0.0000E+00
	$\pm 2.0370E+01$	$\pm 0.0000E+00$
	(3.8078s)	(2.3953s)

doi: 10.1371/journal.pone.0122827.t003

Ab Wahab MN, Nefti-Meziani S, Atyabi A (2015) A Comprehensive Review of Swarm Optimization Algorithms. PLoS ONE 10(5):e0122827. doi:10.1371/journal.pone.0122827

Performance Comparison

Table 5. Benchmark Functions Comparison of mean error (Mean $\pm$ SD) and time (Seconds) on

Function	Several Optimization Techniques.	GA	PSO
Bohachevsky1 (Separable)		0.0000E+00	0.0000E+00
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(6.3516s)	(1.9328s)
Booth (Separable)		0.0000E+00	0.0000E+00
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(2.5125s)	(1.8234s)
Branin (Separable)		3.9789E-01	3.9789E-01
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(5.9844s)	(1.9719s)
Michalewicz5 (Separable)		-1.5651E+00	-1.1906E+00
		$\pm 0.0000E+00$	$\pm 3.3261E-01$
		(2.6859s)	(1.9797s)
Rastrigin (Separable)		5.5900E+01	5.4130E+02
		$\pm 1.4294E+01$	$\pm 1.5969E+01$
		(3.7891s)	(2.9531s)
Shubert (Separable)		-1.2884E+02	-1.8673E+02
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(3.2000s)	(1.9844s)
Ackley (Inseparable)		1.7194E+01	1.6004E+01
		$\pm 7.9083E-01$	$\pm 5.3105E+00$
		(4.0344s)	(2.9844s)
Bohachevsky2 (Inseparable)		0.0000E+00	0.0000E+00
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(5.7203s)	(2.0359s)
Bohachevsky3 (Inseparable)		0.0000E+00	0.0000E+00
		$\pm 0.0000E+00$	$\pm 0.0000E+00$
		(5.5688s)	(1.9438s)

Ab Wahab MN, Nefti-Meziani S, Atyabi A
(2015) A Comprehensive Review of
Swarm Optimization Algorithms. PLoS
ONE 10(5):e0122827.
doi:10.1371/journal.pone.0122827

doi:10.1371/journal.pone.0122827.t005

Performance Comparison: Overall Results on 30 benchmark functions

Table 9. Performance breakdown based on the benchmark functions' characteristics.

Category	Number of functions	GA	ACO	PSO	DE	ABC	GSO	CSA
Being best performing method	30	15	13	19	24	6	0	3
Unimodal Separable (US)	2	0	0	1	1	0	0	0
Unimodal Inseparable (UI)	10	3	3	6	7	2	0	1
Multimodal Separable (MS)	6	4	3	4	5	3	0	2
Multimodal Inseparable (MI)	12	8	7	8	11	1	0	0
Unimodal (U)	12	3	3	7	8	2	0	1
Multimodal (M)	18	12	10	12	16	4	0	2
Separable (S)	8	4	3	5	6	3	0	2
Inseparable (I)	22	11	10	14	18	3	0	1

doi:10.1371/journal.pone.0122827.t009

Ab Wahab MN, Nefti-Meziani S, Atyabi A (2015) A Comprehensive Review of Swarm Optimization Algorithms. PLoS ONE 10(5):e0122827. doi:10.1371/journal.pone.0122827

Things to consider when deciding on which evolutionary method to use

- Based on performance with benchmark functions, PSO and DE are known to be the most effective optimization techniques.
- PSO is known to be flexible in its design and when combined with macroscopic and microscopic modeling, it has the ability to fit the problem at hand.
- How to model the evolutionary method is highly effected by the user's experience. An ill modeled EA method can be computationally intensive, memory intensive, and achieve lower than expected performance.

Lecture 9: Search Optimization- Part II

Simulated Annealing, Tabu Search & Ant Colony Optimization

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

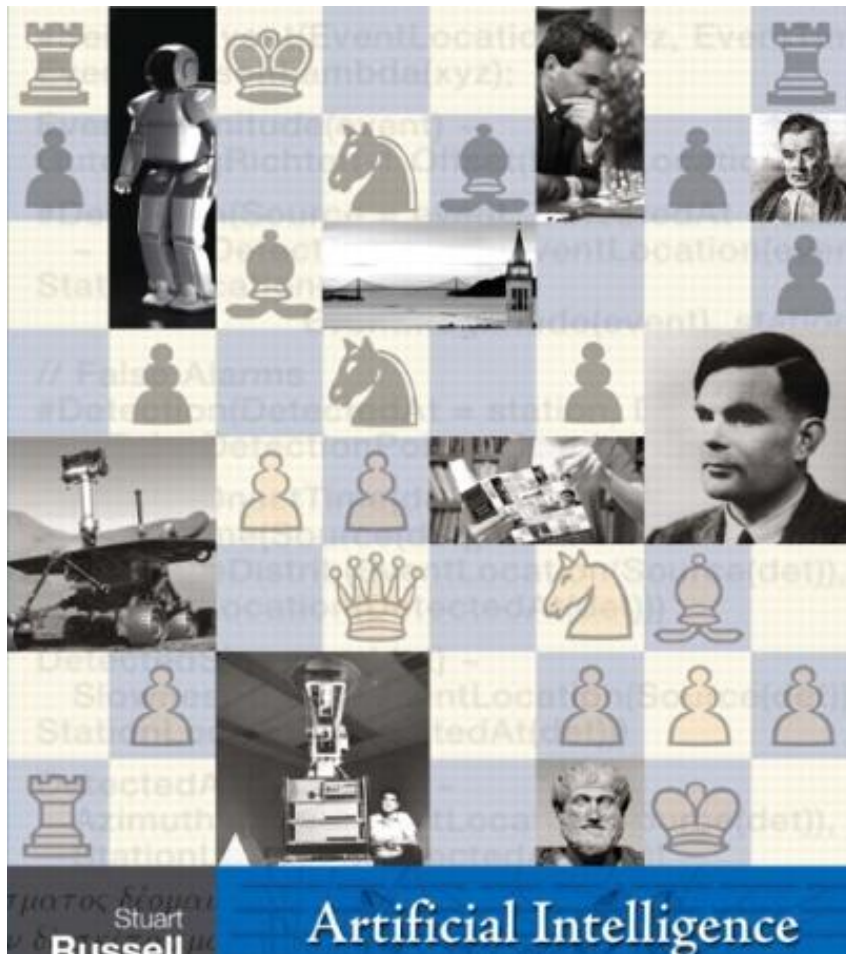
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Disclosure

- We are using materials from Prof. Panagiotis Repoussis's lecture on metaheuristic algorithms in Stevens Institute of Technology
- We are also using Prof. Jugal Kalita's materials and lecture slides on Ant Colony Optimization

References

- Talbi E.-G., 2009. *Metaheuristics: from design to implementation*, Vol. 74, John Wiley & Sons
- Blum C. and Roli A., 2003. Metaheuristics in combinatorial optimization: Overview and conceptual comparison, *ACM Computing Surveys*, 35(3):268-308
- Back T., Hammel U. and Schwefel H.-P., 2002. Evolutionary computation: comments on the history and current state, *IEEE Transactions on Evolutionary Computation*, 1(1):3-17

Introduction

- *Heuristics*
 - “to find” (from ancient Greek “εὕρισκειν”)
- *Meta-*
 - An abstraction from another concept
 - ***beyond, in an upper level***
 - ***used to complete or add to***
 - E.g., meta-data = “data about data”
- *Metaheuristics*
 - “A heuristic around heuristics”

Introduction

- Some popular frameworks
 - Genetic Algorithms
 - Simulated Annealing
 - Tabu Search
 - Scatter Search
 - Ant Colony Optimization
 - Particle Swarm Optimization
 - Iterated Local Search
 - Variable Neighborhood Search
 - Adaptive Memory Programming
 - ...

Introduction

- Metaheuristics...
 - ...can address both discrete- and continuous-domain optimization problems
 - ...are strategies that “guide” the search process
 - ...range from simple local search procedures to complex adaptive learning processes

Introduction

- Metaheuristics...
 - ...efficiently explore the search space in order to find good (near-optimal) feasible solutions
 - ...provide no guarantee of global or local optimality
 - ...are agnostic to the unexplored feasible space (i.e., no “bound” information)
 - ...lack a metric of “goodness” of solution (often stop due to an external time or iteration limit)
 - ...are not based on some algebraic model

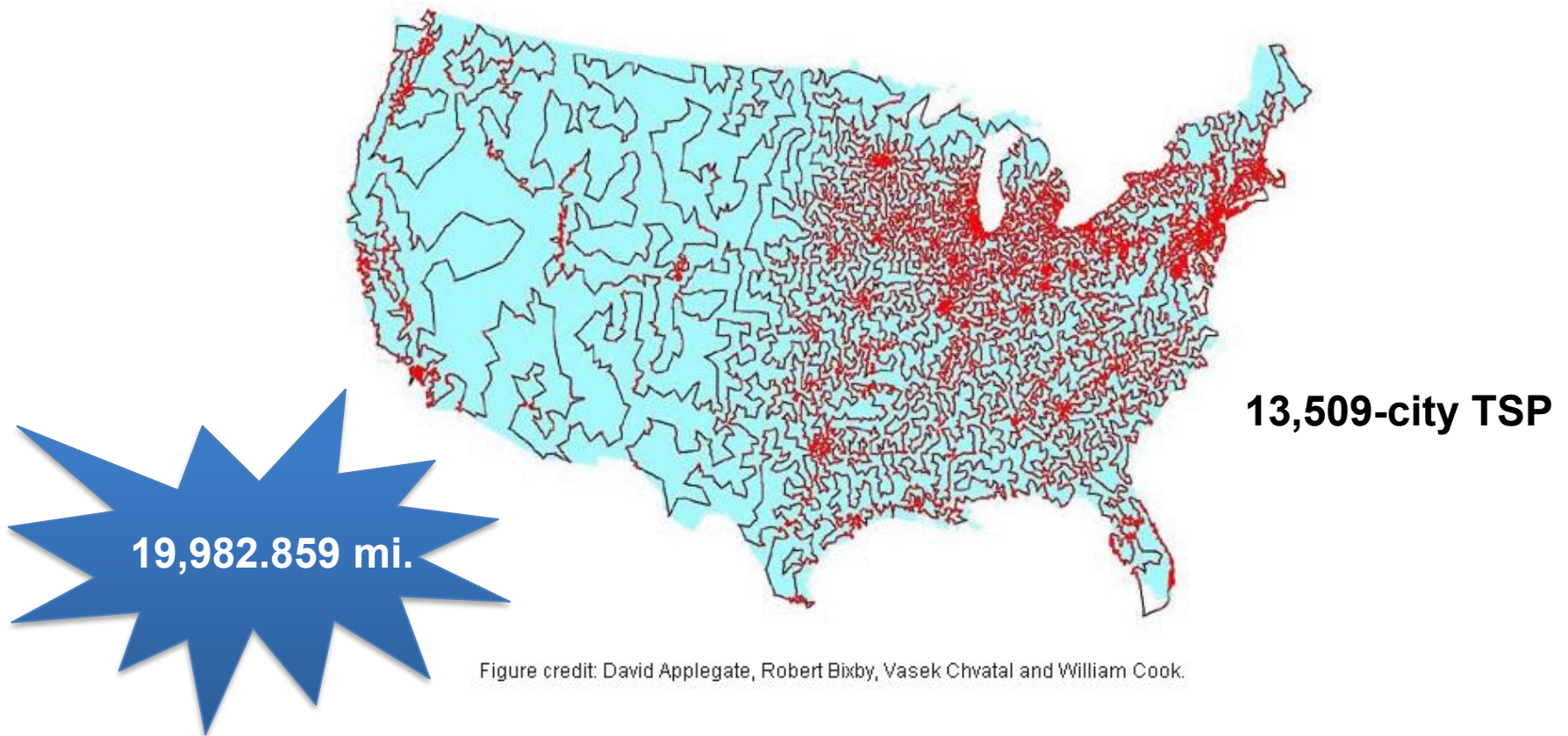
unlike exact methods!

- ...are often used in conjunction with an exact method
 - ***E.g., use metaheuristic to provide upper bounds***
 - ***E.g., use restricted MILP as “local heuristic”***

Introduction

- Let's consider the problem of find the best visiting sequence (route) to serve 14 customers.
- How many possible routes exists?
 - $(n-1)! = (15-1)! = 14! = 8,7178 \times 10^{10} = 88$ billion solutions
 - If a PC can check about 1 million solution per second, then we need $8,8 \times 10^{10}$ routes / 10^6 routes/ sec = 88.000 sec or about 24,44 hours to check them all and find the best!

COMBINATORIAL EXPLOSION



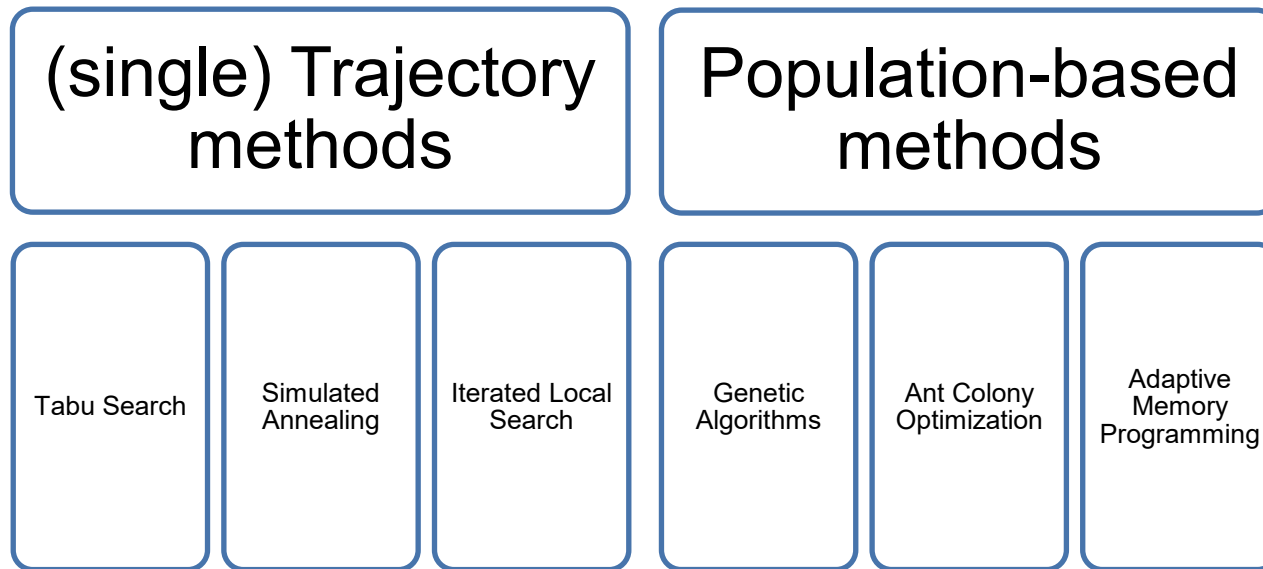
- What type of algorithm would you pick to solve a problem with $10^{49,933}$ feasible solutions?

Two basic definitions

- A **neighborhood** $C(s)$ around a solution s is a function that returns a subset of solutions that are somehow close or related to s
 - implies a “direction of search”
 - multiple neighborhoods can be defined for a solution s
 - could be exhaustive or not (if not, it could actually be random)
- A **move** $s \leftarrow s'$ constitutes a choice of a solution $s' \in C(s)$

CLASSIFICATION

- Based on whether the search evolves one or multiple solutions



- Trend towards population-based methods, with integration of proven ideas from trajectory methods

CLASSIFICATION #5

Trajectory (S-metaheuristics)

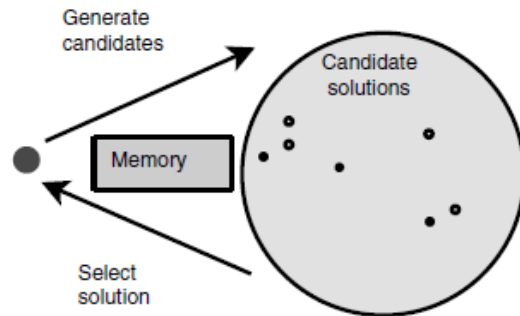


FIGURE 2.1 Main principles of single-based metaheuristics.

Population-based (P-metaheuristics)

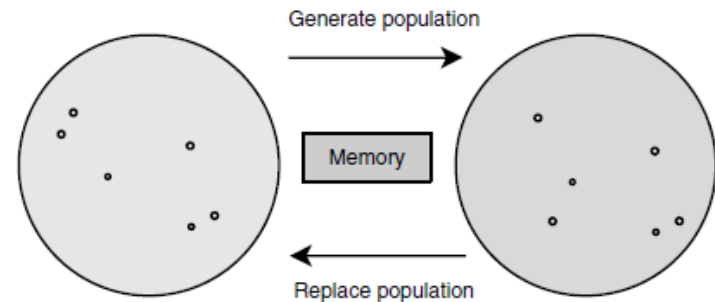


FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

```

Input: Initial solution  $s_0$ .
 $t = 0$ ;
Repeat
  /* Generate candidate solutions (partial or complete neighborhood) from  $s_t$  */
  Generate( $C(s_t)$ );
  /* Select a solution from  $C(s_t)$  to replace the current solution  $s_t$  */
   $s_{t+1} = \text{Select}(C(s_t))$ ;
   $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution found.
  
```

Algorithm 3.1 High-level template of P-metaheuristics.

```

 $P = P_0$ ; /* Generation of the initial population */
 $t = 0$ ;
Repeat
  Generate( $P'_t$ ); /* Generation a new population */
   $P_{t+1} = \text{Select-Population}(P_t \cup P'_t)$ ; /* Select new population */
   $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution(s) found.
  
```

TRAJECTORY METHODS

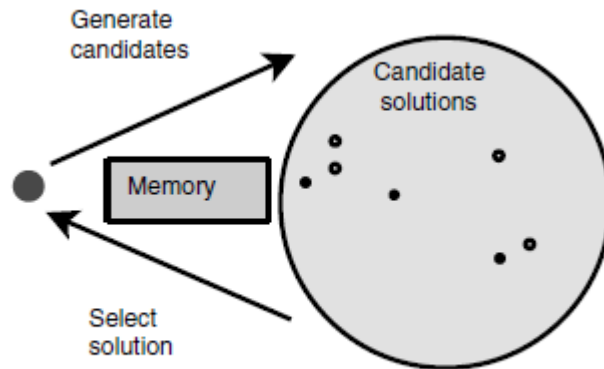


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .

$t = 0$;

Repeat

/* Generate candidate solutions (partial or complete neighborhood) from s_t */
Generate($C(s_t)$);

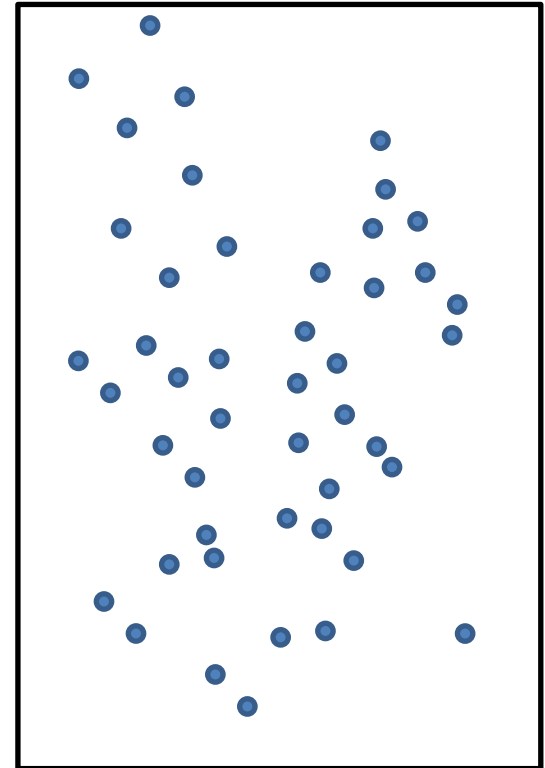
/* Select a solution from $C(s)$ to replace the current solution s_t */

$s_{t+1} = \text{Select}(C(s_t))$;

$t = t + 1$;

Until Stopping criteria satisfied

Output: Best solution found.



TRAJECTORY METHODS

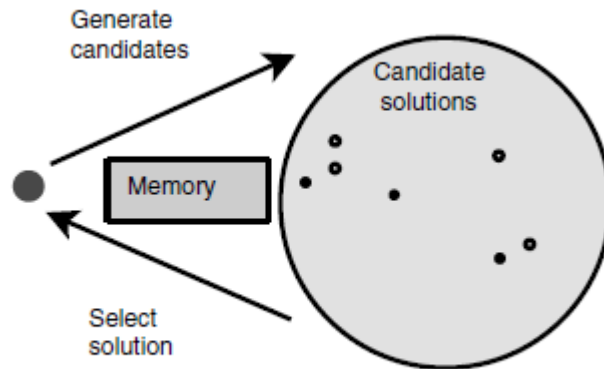


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .

$t = 0$;

Repeat

/* Generate candidate solutions (partial or complete neighborhood) from s_t */
 Generate($C(s_t)$);

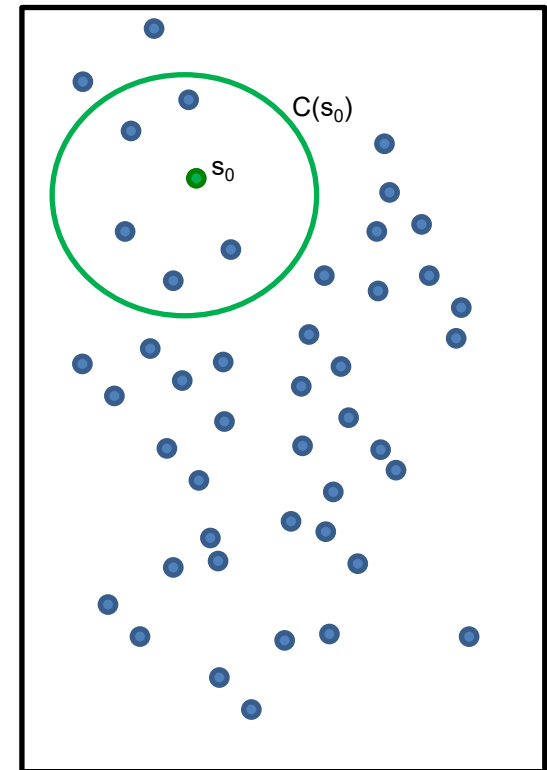
/* Select a solution from $C(s)$ to replace the current solution s_t */

$s_{t+1} = \text{Select}(C(s_t))$;

$t = t + 1$;

Until Stopping criteria satisfied

Output: Best solution found.



TRAJECTORY METHODS

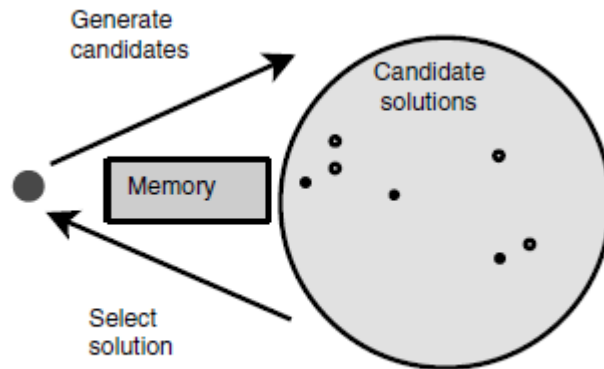


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .

$t = 0$;

Repeat

 /* Generate candidate solutions (partial or complete neighborhood) from s_t */
 Generate($C(s_t)$) ;

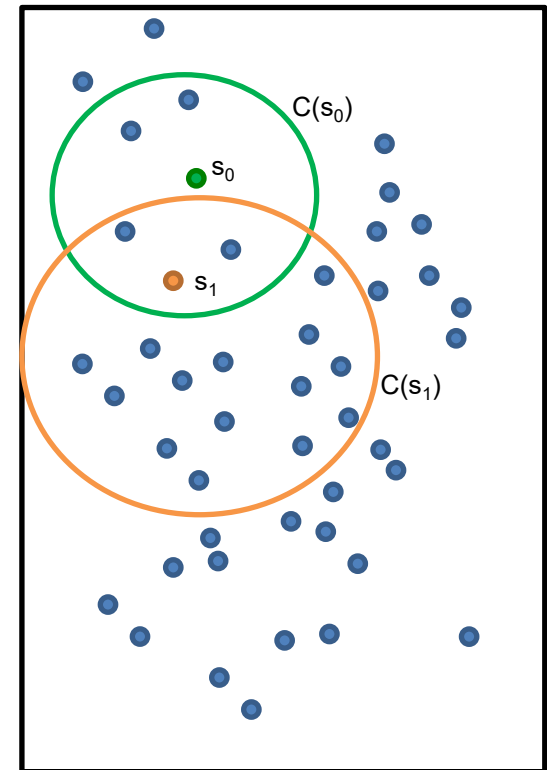
 /* Select a solution from $C(s)$ to replace the current solution s_t */

$s_{t+1} = \text{Select}(C(s_t))$;

$t = t + 1$;

Until Stopping criteria satisfied

Output: Best solution found.



TRAJECTORY METHODS

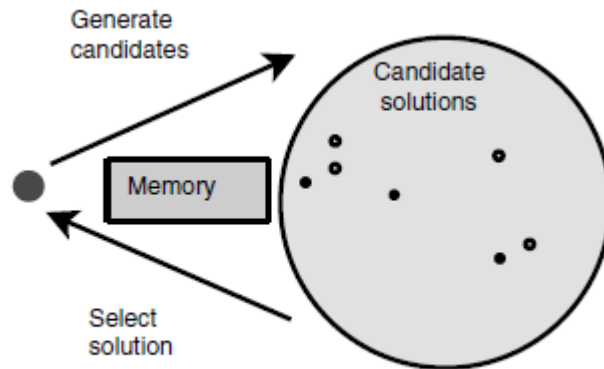


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .

$t = 0$;

Repeat

/* Generate candidate solutions (partial or complete neighborhood) from s_t */
 $\text{Generate}(C(s_t))$;

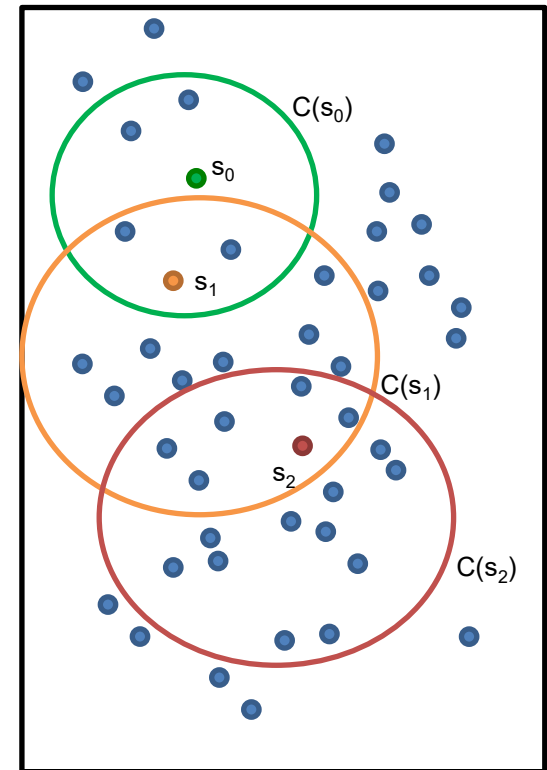
/* Select a solution from $C(s)$ to replace the current solution s_t */

$s_{t+1} = \text{Select}(C(s_t))$;

$t = t + 1$;

Until Stopping criteria satisfied

Output: Best solution found.



Stop because of no improvement in region $C(s_2)$

TRAJECTORY METHODS

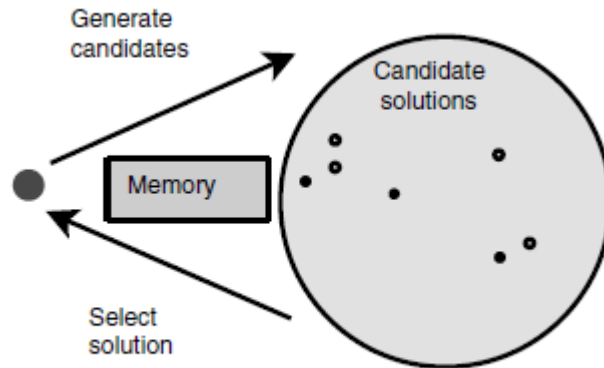
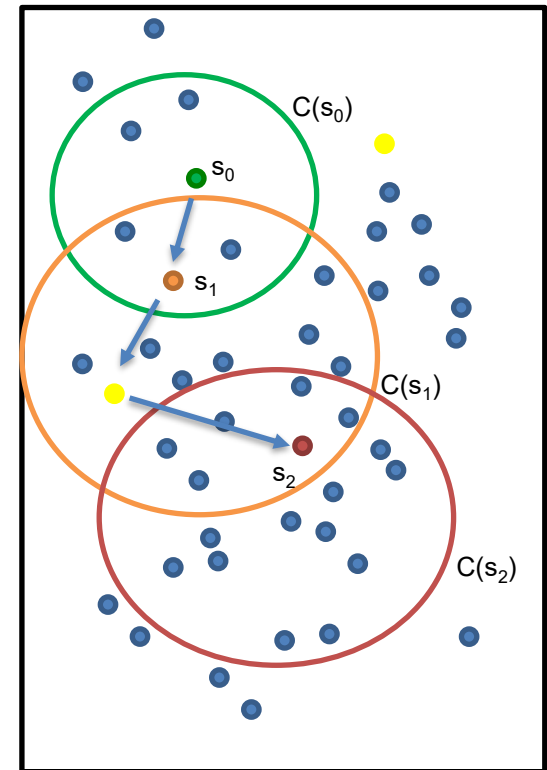


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .
 $t = 0$;
Repeat
 /* Generate candidate solutions (partial or complete neighborhood) from s_t */
 Generate($C(s_t)$) ;
 /* Select a solution from $C(s)$ to replace the current solution s_t */
 $s_{t+1} = \text{Select}(C(s_t))$;
 $t = t + 1$;
Until Stopping criteria satisfied
Output: Best solution found.



Turns out two global optima in this problem, but none was identified

POPULATION-BASED METHODS

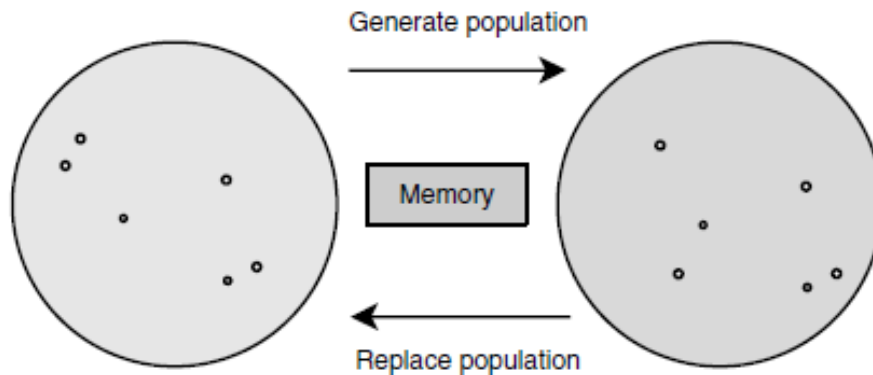
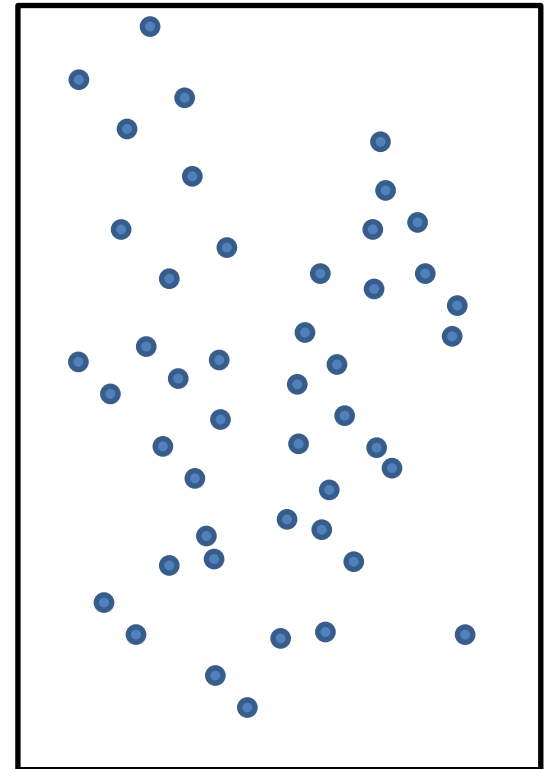


FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```

 $P = P_0$ ; /* Generation of the initial population */
 $t = 0$ ;
Repeat
    Generate( $P_t'$ ); /* Generation a new population */
     $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */
     $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution(s) found.
    
```



POPULATION-BASED METHODS

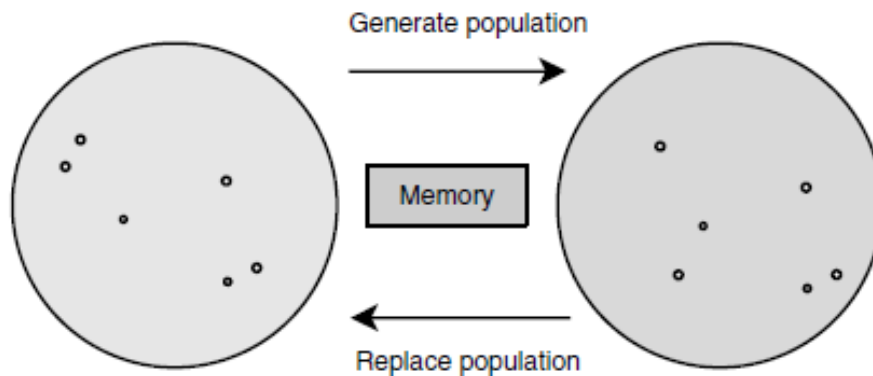


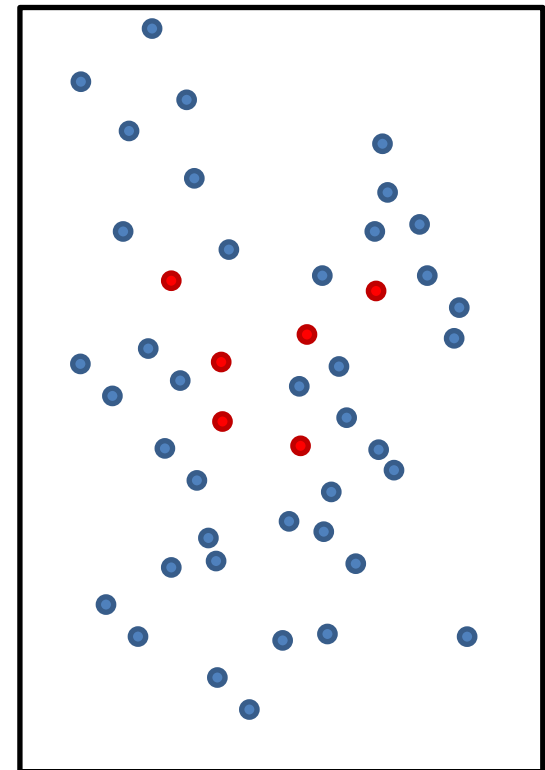
FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```

 $P = P_0$ ; /* Generation of the initial population */
 $t = 0$ ;
Repeat
    Generate( $P_t'$ ); /* Generation a new population */
     $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */
     $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution(s) found.
    
```

0th population



POPULATION-BASED METHODS

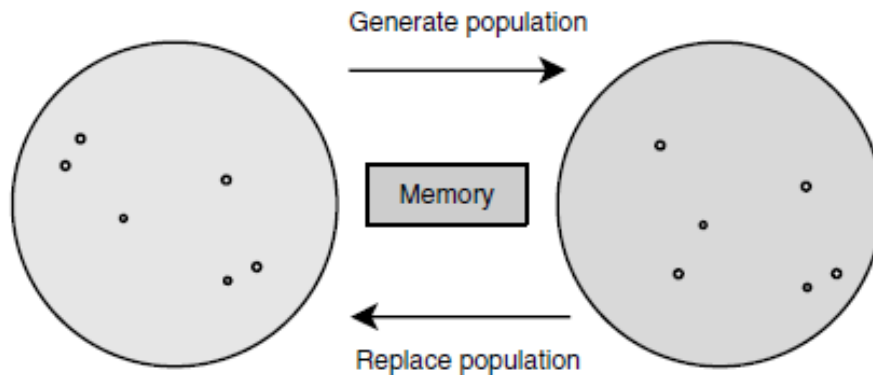
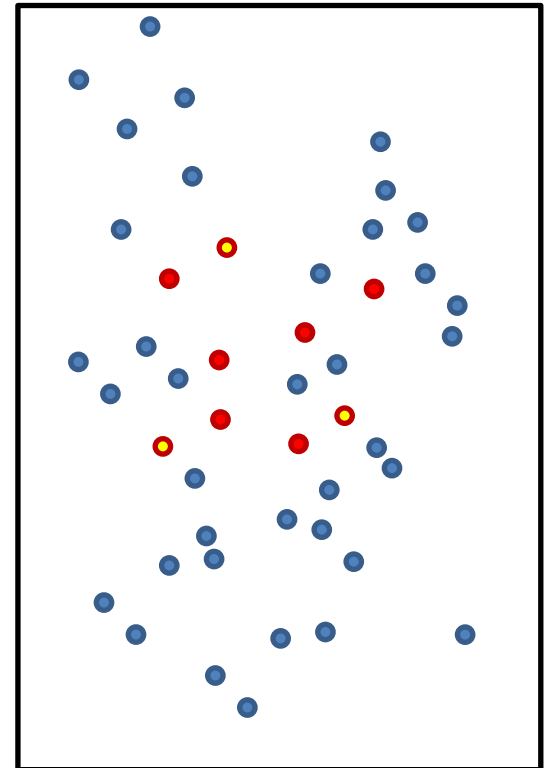


FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```
 $P = P_0$ ; /* Generation of the initial population */  
 $t = 0$  ;  
Repeat  
  Generate( $P_t'$ ); /* Generation a new population */  
   $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */  
   $t = t + 1$ ;  
Until Stopping criteria satisfied  
Output: Best solution(s) found.
```

Get some new points



POPULATION-BASED METHODS

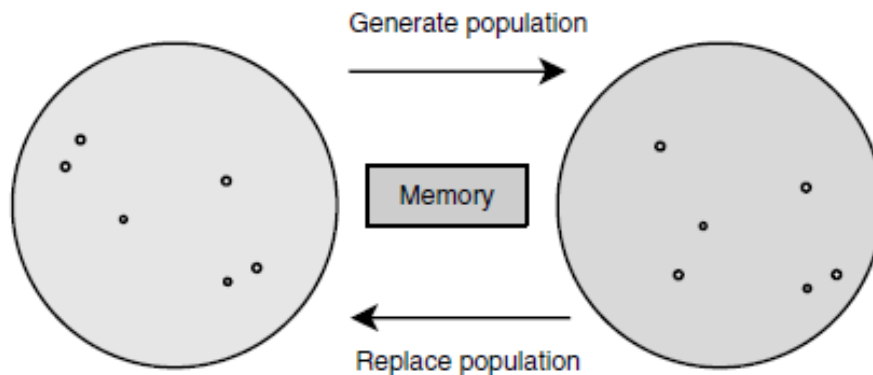


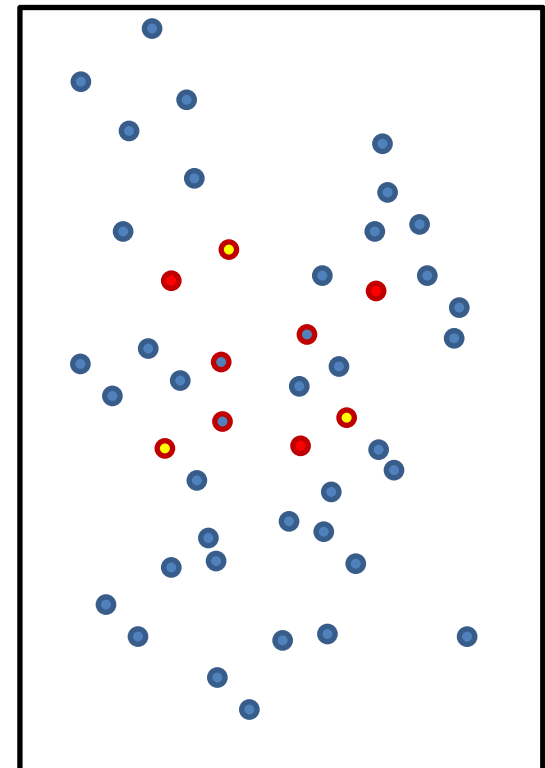
FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```

 $P = P_0$ ; /* Generation of the initial population */
 $t = 0$ ;
Repeat
    Generate( $P_t'$ ); /* Generation a new population */
     $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */
     $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution(s) found.
    
```

Drop some old points



POPULATION-BASED METHODS

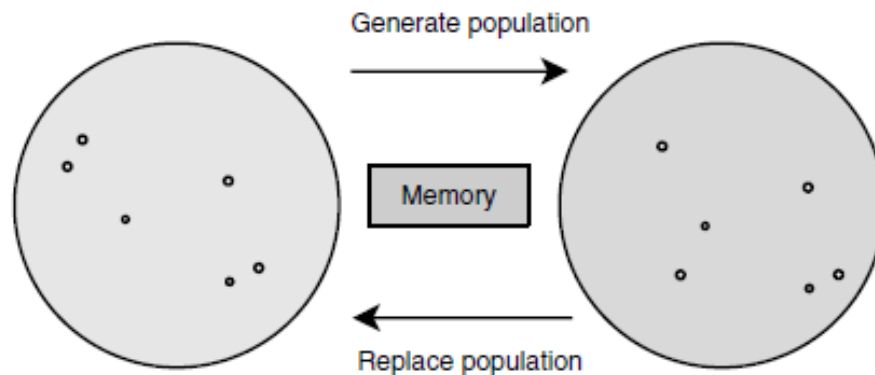


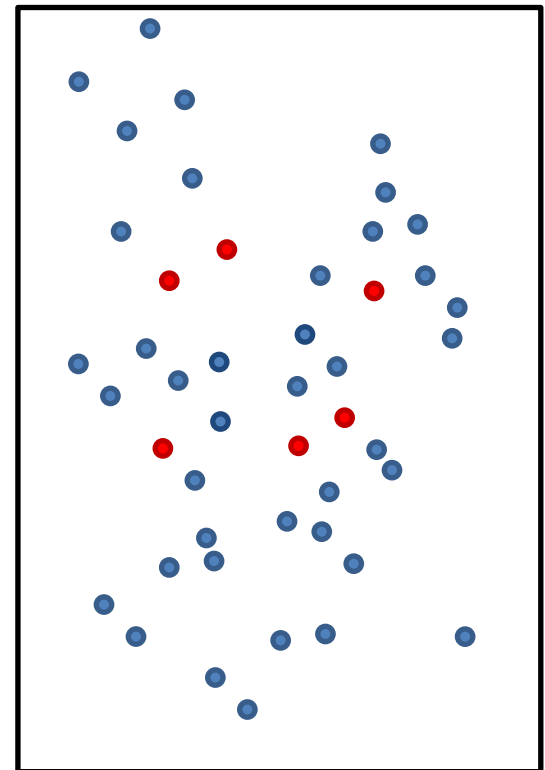
FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```

 $P = P_0$ ; /* Generation of the initial population */
 $t = 0$ ;
Repeat
    Generate( $P_t'$ ); /* Generation a new population */
     $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */
     $t = t + 1$ ;
Until Stopping criteria satisfied
Output: Best solution(s) found.
    
```

1st population



POPULATION-BASED METHODS

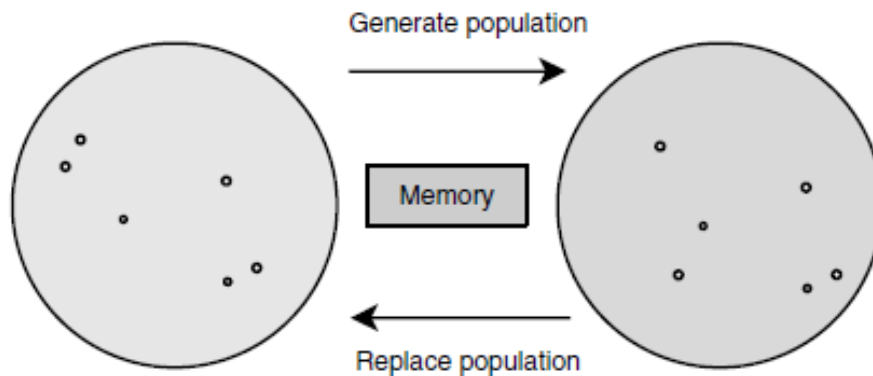
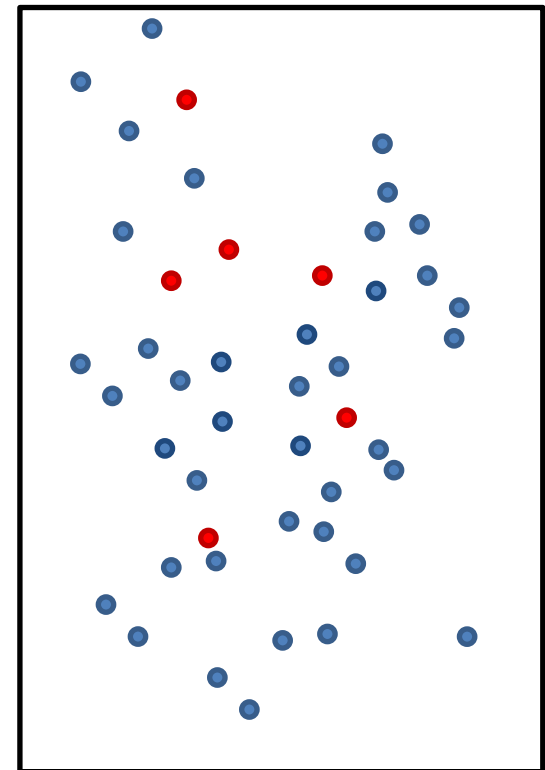


FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

$P = P_0$; /* Generation of the initial population */
 $t = 0$;
Repeat
 Generate(P_t'); /* Generation a new population */
 $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$; /* Select new population */
 $t = t + 1$;
Until Stopping criteria satisfied
Output: Best solution(s) found.

2nd population



POPULATION-BASED METHODS

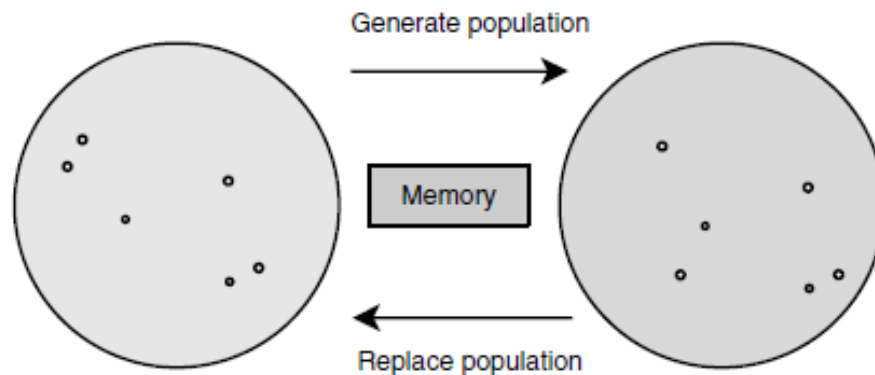
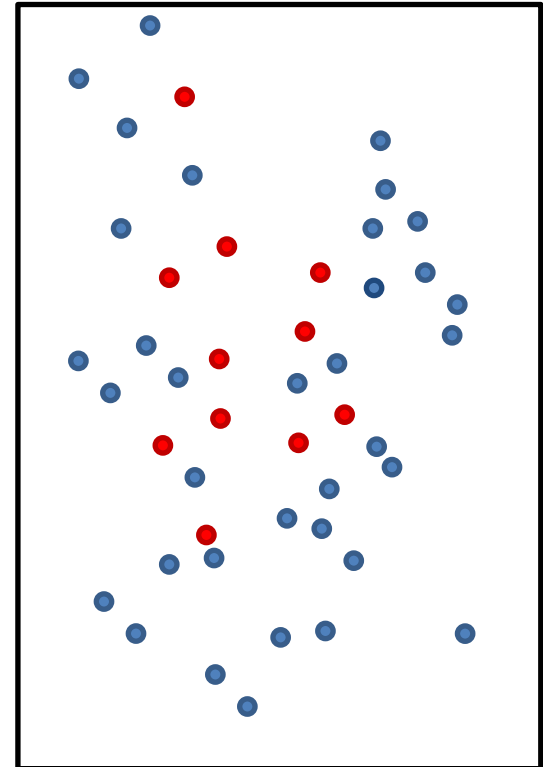


FIGURE 3.1 Main principles of P-metaheuristics.

Algorithm 3.1 High-level template of P-metaheuristics.

```
 $P = P_0$ ; /* Generation of the initial population */  
 $t = 0$  ;  
Repeat  
  Generate( $P_t'$ ); /* Generation a new population */  
   $P_{t+1} = \text{Select-Population}(P_t \cup P_t')$ ; /* Select new population */  
   $t = t + 1$ ;  
Until Stopping criteria satisfied  
Output: Best solution(s) found.
```

All points sampled



Again, optimum may or may not have been sampled

- Typically, the incumbent always remains in the population, so need only focus on last generation

CLASSIFICATION #5

Trajectory (S-metaheuristics)

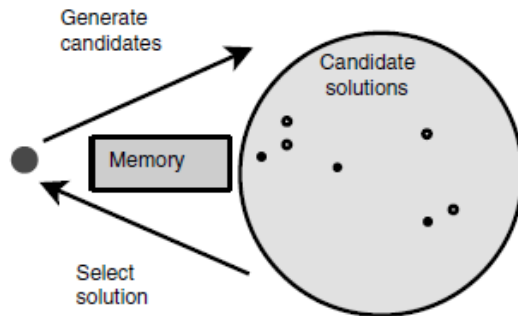


FIGURE 2.1 Main principles of single-based metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

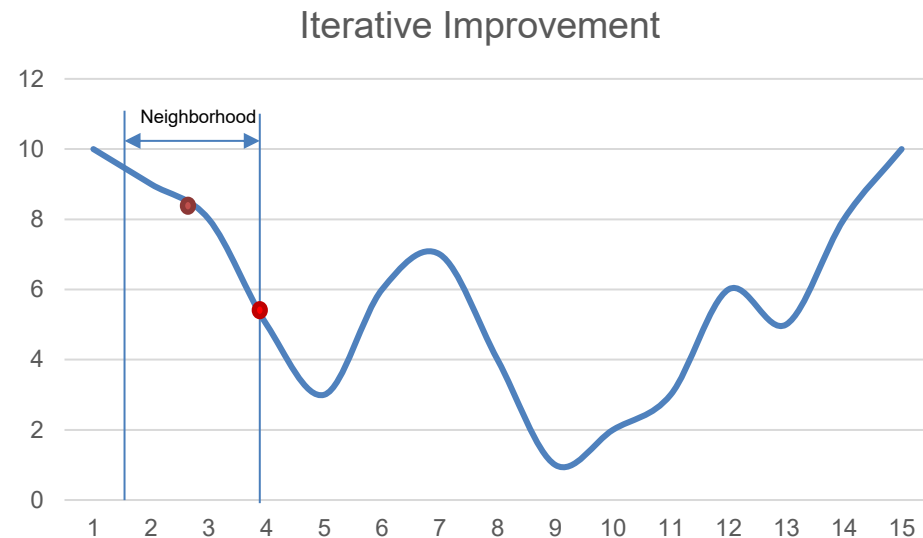
```
Input: Initial solution  $s_0$ .  
 $t = 0$ ;  
Repeat  
  /* Generate candidate solutions (partial or complete neighborhood) from  $s_t$  */  
  Generate( $C(s_t)$ );  
  /* Select a solution from  $C(s)$  to replace the current solution  $s_t$  */  
   $s_{t+1} = \text{Select}(C(s_t))$ ;  
   $t = t + 1$ ;  
Until Stopping criteria satisfied  
Output: Best solution found.
```

- Basic Local Search (3192 BC)
- Simulated Annealing (1983)
- Tabu Search (1986)
- GRASP (1989)
- Guided Local Search (1997)
- Variable Neighborhood Search (1999)
- Iterated Local Search (1999)

BASIC LOCAL SEARCH

```
s ← GenerateInitialSolution()
repeat
  s ← Improve(N(s))
until no improvement is possible
```

Fig. 1. Algorithm: Iterative Improvement.

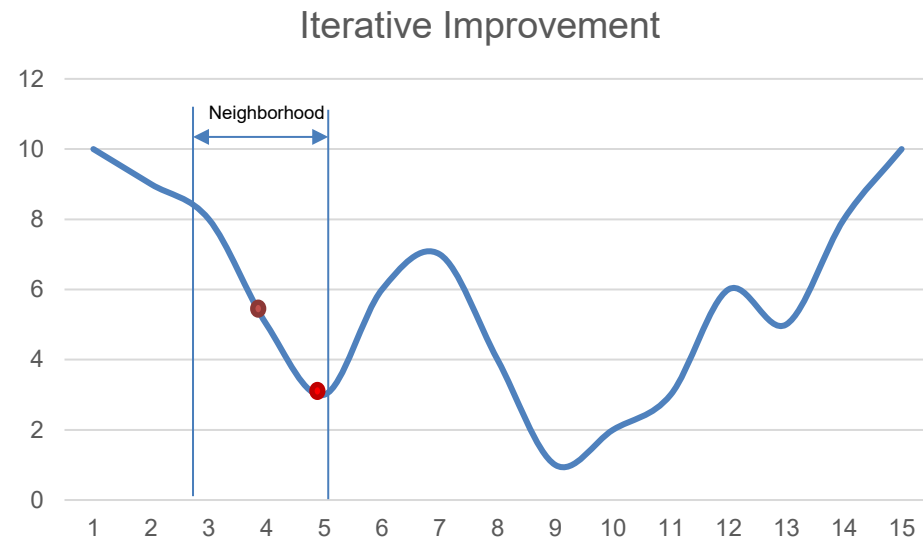


Improve() can be implemented as “first improvement” or “best improvement” or anything in between

BASIC LOCAL SEARCH

```
 $s \leftarrow \text{GenerateInitialSolution}()$   
repeat  
   $s \leftarrow \text{Improve}(\mathcal{N}(s))$   
until no improvement is possible
```

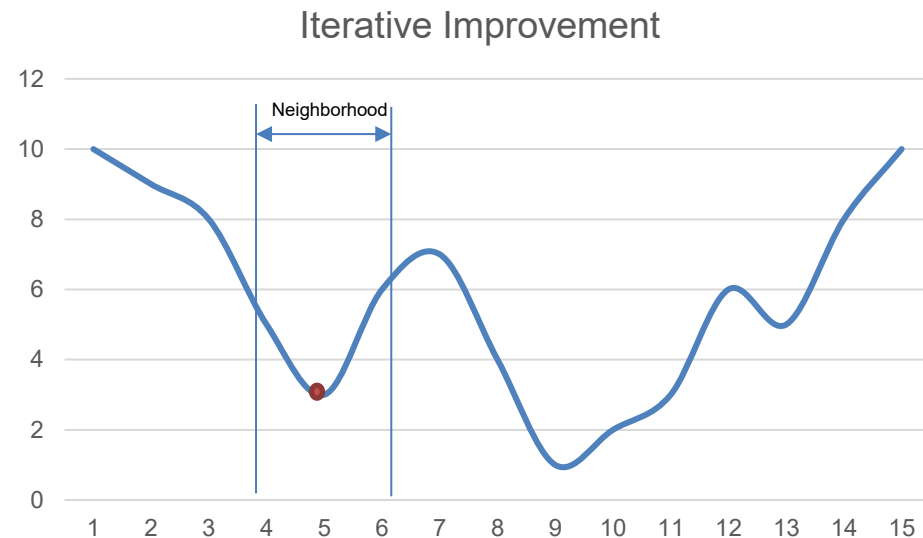
Fig. 1. Algorithm: Iterative Improvement.



BASIC LOCAL SEARCH

```
s ← GenerateInitialSolution()
repeat
  s ← Improve(N(s))
until no improvement is possible
```

Fig. 1. Algorithm: Iterative Improvement.



Local optimum found, cannot improve!

How to escape the local optimum “trap”?

ESCAPING LOCAL OPTIMA

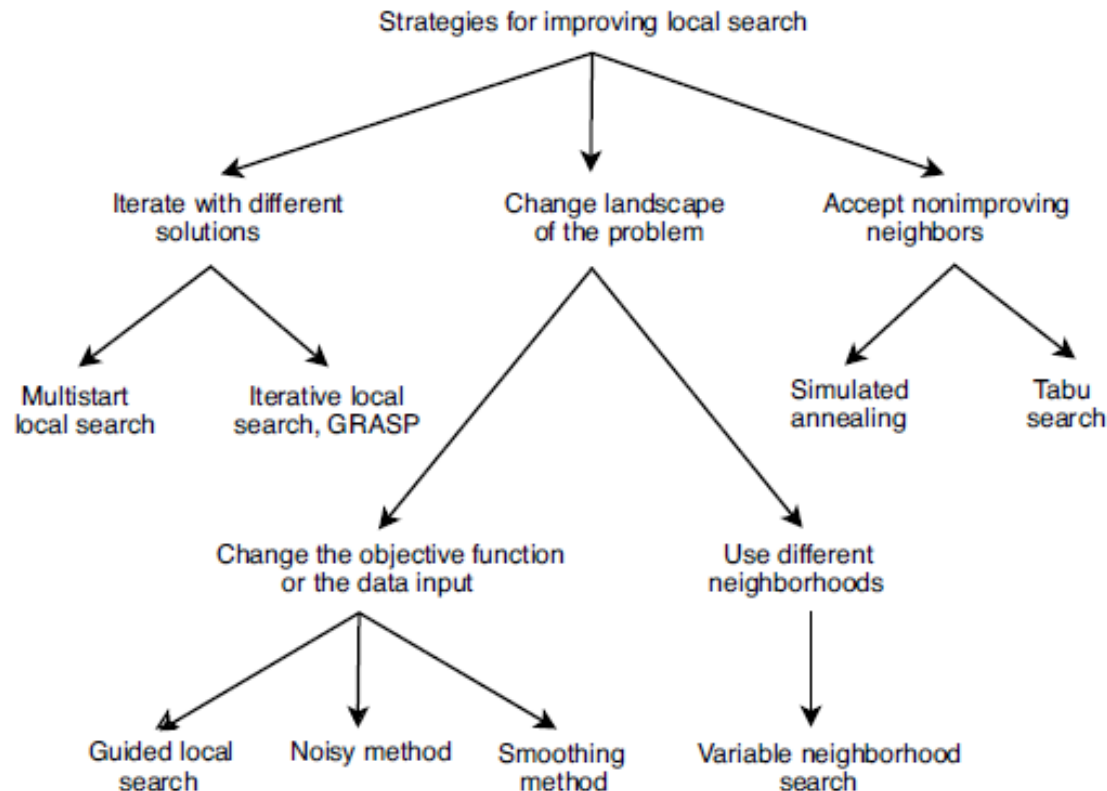


FIGURE 2.24 S-metaheuristic family of algorithms for improving local search and escaping from local optima.

INTENSIFY & DIVERSIFY

- Merriam-Webster:

Intensify

- to become stronger or more extreme; to become more intense

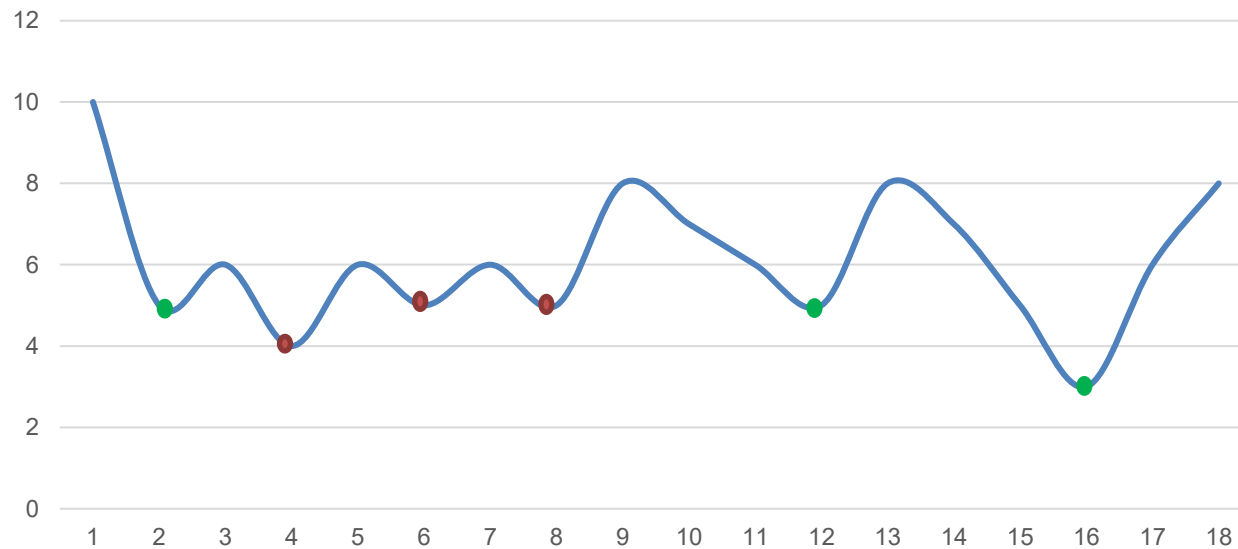
Diversify

- to change (something) so that it has more different kinds of people or things

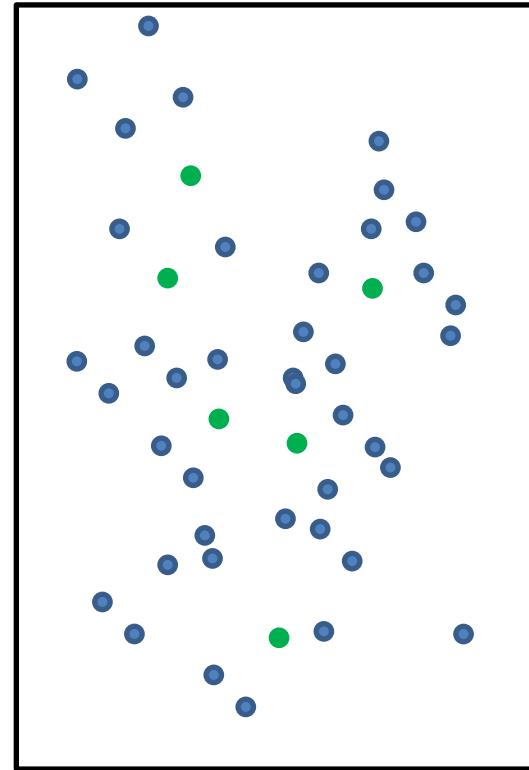
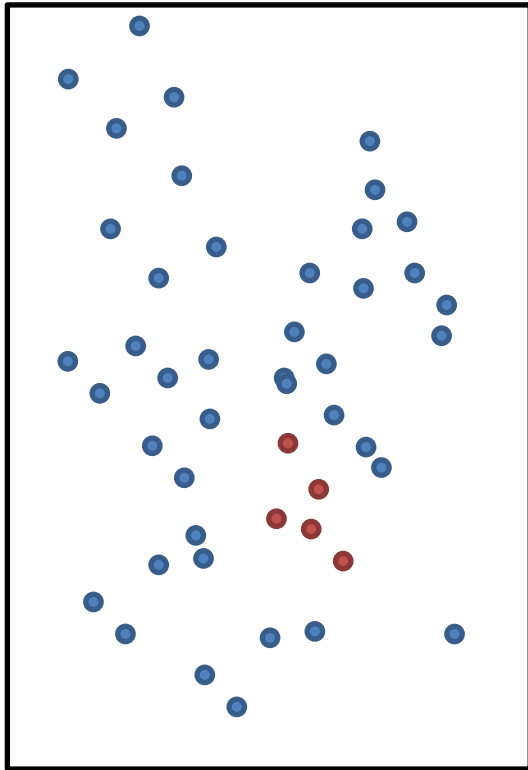
INTENSIFY & DIVERSIFY

- Every metaheuristic aspires to do both
- Many have parameters to control this balance
- Often, these parameters can be changed dynamically during search
- Initially, focus is on diversification; later, on intensification (and cycles back)

INTENSIFY & DIVERSIFY



INTENSIFY & DIVERSIFY



TABU SEARCH

- **Taboo** (*English*): prohibited, disallowed, forbidden
- **Tabu** (*Fijian*): forbidden to use due to being sacred and/or of supernatural powers
- ...related to similar Polynesian words
tapu (*Tongan*), tapu (*Maori*), kapu (Hawaiian)

TABU SEARCH

```
s ← GenerateInitialSolution()
TabuList ← ∅
while termination conditions not met do
    s ← ChooseBestOf( $\mathcal{N}(s) \setminus \textit{TabuList}$ )
    Update(TabuList)
endwhile
```

Fig. 3. Algorithm: Simple Tabu Search (TS).

- Used mainly for discrete problems
- The tabu list constitutes “short-term memory”
- Size of tabu list (“tenure”) is finite
- Since solutions in tabu list are off-limits, it helps
 - escape local minima by forcing uphill moves (if no improving move available)
 - avoid cycling (up to the period induced by tabu list size)
- Solutions enter and leave the list in a FIFO order (usually)

TABU SEARCH

```
s ← GenerateInitialSolution()
TabuList ← ∅
while termination conditions not met do
    s ← ChooseBestOf( $\mathcal{N}(s) \setminus \textit{TabuList}$ )
    Update(TabuList)
endwhile
```

Fig. 3. Algorithm: Simple Tabu Search (TS).

- Small tenure localizes search (intensification)
- Large tenure forces exploration of wider space (diversification)
- Tenure can change dynamically during search
- Size of tenure is a form of “long-term memory”

TABU SEARCH

```
s ← GenerateInitialSolution()
TabuList ← ∅
while termination conditions not met do
    s ← ChooseBestOf( $\mathcal{N}(s) \setminus \textit{TabuList}$ )
    Update(TabuList)
endwhile
```

Fig. 3. Algorithm: Simple Tabu Search (TS).

- Storing complete solutions is inefficient
 - implementation perspective (storage, comparisons)
 - algorithm perspective (largely similar solutions offer no interesting information)
- Tabu search usually stores “solution attributes”
 - solution components or solution differences (“moves”)

```

Input:  $TabuList_{size}$ 
Output:  $S_{best}$ 
 $S_{best} \leftarrow \text{ConstructInitialSolution}()$ 
 $TabuList \leftarrow \emptyset$ 
While ( $\neg \text{StopCondition}()$ )
     $CandidateList \leftarrow \emptyset$ 
    For ( $S_{candidate} \in S_{best} \text{ neighborhood}$ )
        If ( $\neg \text{ContainsAnyFeatures}(S_{candidate}, TabuList)$ )
             $CandidateList \leftarrow S_{candidate}$ 
        End
    End
     $S_{candidate} \leftarrow \text{LocateBestCandidate}(CandidateList)$ 
    If ( $\text{Cost}(S_{candidate}) \leq \text{Cost}(S_{best})$ )
         $S_{best} \leftarrow S_{candidate}$ 
         $TabuList \leftarrow \text{FeatureDifferences}(S_{candidate}, S_{best})$ 
        While ( $TabuList > TabuList_{size}$ )
            DeleteFeature( $TabuList$ )
        End
    End
End
Return ( $S_{best}$ )

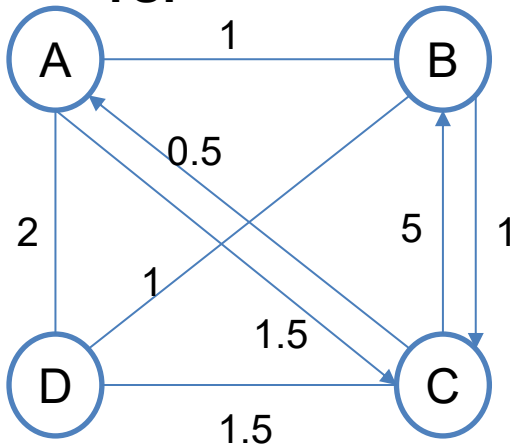
```

Pseudocode for Tabu Search.

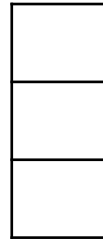
http://www.cleveralgorithms.com/nature-inspired/stochastic/tabu_search.html#targetText=Tabu%20Search%20is%20a%20Global,Search%20and%20Parallel%20Tabu%20Search.

TABU SEARCH

**Asymmetric
TSP**



**Tabu List
(tenure = 3)**



Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

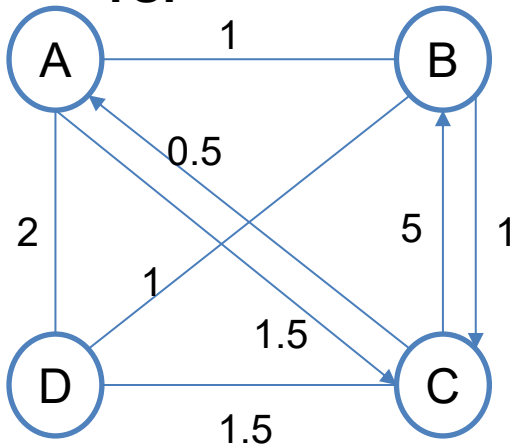
solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

A
B
B
C
C
D
D
A

B	A	C	D	B
---	---	---	---	---

5.0

A	C	B	D	A
---	---	---	---	---

9.5

A	B	D	C	A
---	---	---	---	---

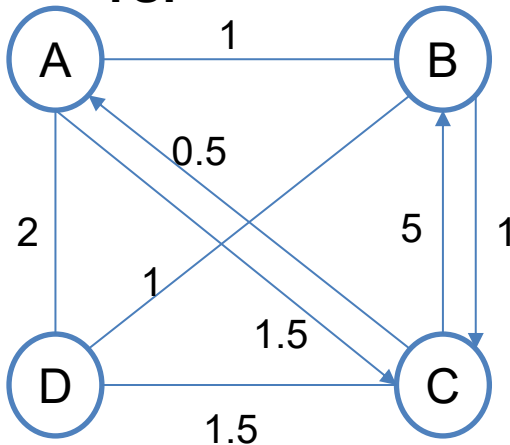
4.0

D	B	C	A	D
---	---	---	---	---

4.5

TABU SEARCH

**Asymmetric
TSP**



**Tabu List
(tenure = 3)**

Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

A	B	D	C	A
---	---	---	---	---

4.0

B	A	C	D	B
---	---	---	---	---

5.0

A	C	B	D	A
---	---	---	---	---

9.5

A	B	D	C	A
---	---	---	---	---

4.0

D	B	C	A	D
---	---	---	---	---

4.5

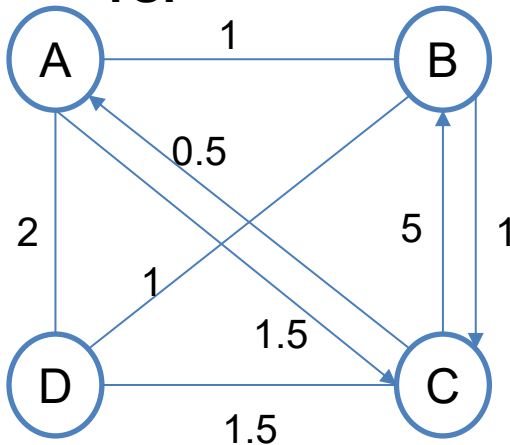
solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

C
D

Solution Trajectory

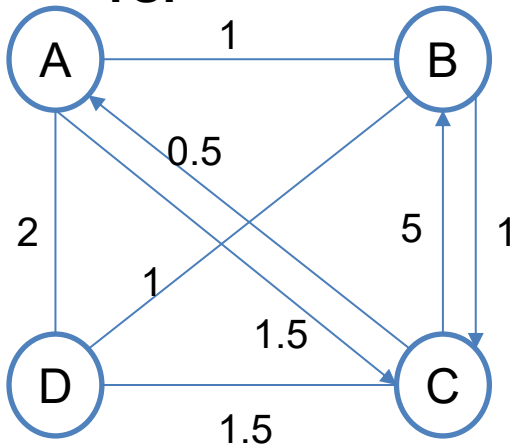
A	B	C	D	A
A	B	D	C	A

5.5

4.0

TABU SEARCH

Asymmetric TSP



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

C
D

A
B
B
D
D
C
C
C
A

Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

A	B	D	C	A
---	---	---	---	---

4.0

B	A	D	C	A
---	---	---	---	---

5.0

A	D	B	C	A
---	---	---	---	---

4.5

A	B	C	D	A
---	---	---	---	---

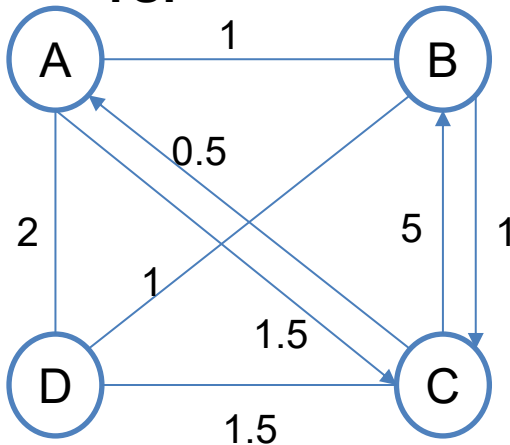
5.5

C	B	D	A	C
---	---	---	---	---

9.5

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

B
D
C
D

A
B
B
D
D
C
C
A

Solution Trajectory

A	B	C	D	A
A	B	D	C	A
A	D	B	C	A

5.5

4.0

4.5

B	A	D	C	A
A	D	B	C	A

5.0

4.5

A	B	C	D	A
---	---	---	---	---

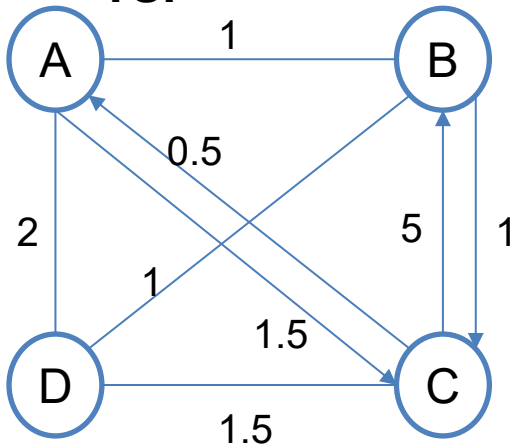
5.5

C	B	D	A	C
---	---	---	---	---

9.5

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

B
D
C
D

A
D
D
B
B
C
C
A

Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

A	B	D	C	A
---	---	---	---	---

4.0

A	D	B	C	A
---	---	---	---	---

4.5

D	A	B	C	D
---	---	---	---	---

5.5

A	B	D	C	A
---	---	---	---	---

4.0

A	D	C	B	A
---	---	---	---	---

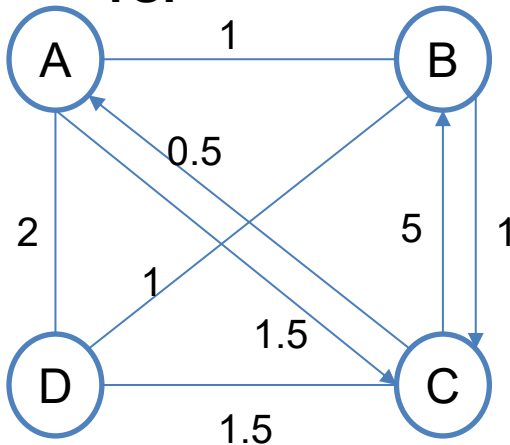
9.5

C	D	B	A	C
---	---	---	---	---

5.0

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

B
D
C
D

A
D
D
B
B
C
C
A

Solution Trajectory

A	B	C	D	A
A	B	D	C	A
A	D	B	C	A

5.5

4.0

4.5

D	A	B	C	D
A	B	D	C	A
A	D	C	B	A
C	D	B	A	C

5.5

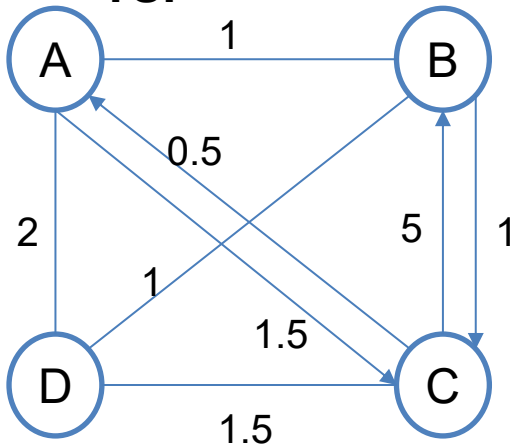
4.0

9.5

5.0

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

C
A
B
D
C
D

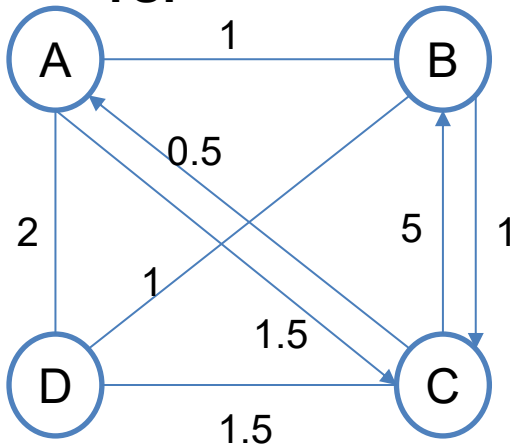
C
D
D
B
B
A
A
C

Solution Trajectory

A	B	C	D	A	5.5
A	B	D	C	A	4.0
A	D	B	C	A	4.5
C	D	B	A	C	5.0
D	C	B	A	D	9.5
C	B	D	A	C	9.5
C	D	A	B	C	5.5
A	D	B	C	A	4.5

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

C
A
B
D
C
D

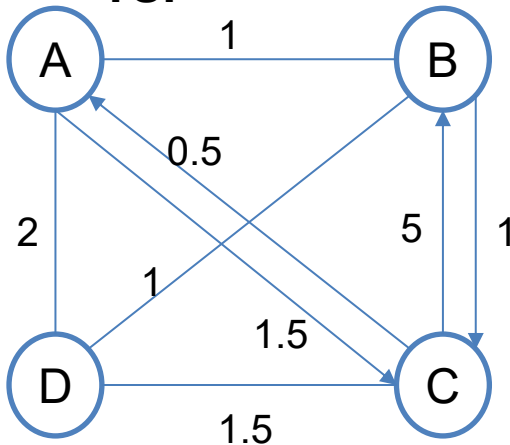
C
D
D
B
B
A
A
C

Solution Trajectory

A	B	C	D	A	5.5
A	B	D	C	A	4.0
A	D	B	C	A	4.5
C	D	B	A	C	5.0
D	C	B	A	D	9.5
C	B	D	A	C	9.5
C	D	A	B	C	5.5
A	D	B	C	A	4.5

TABU SEARCH

**Asymmetric
TSP**



solution = {tour}

move = {swap consecutive pair}

attributes = {moves}

**Tabu List
(tenure = 3)**

B
A
C
A
B
D

C
D
D
A
A
B
B
C

Solution Trajectory

A	B	C	D	A
---	---	---	---	---

5.5

A	B	D	C	A
---	---	---	---	---

4.0

A	D	B	C	A
---	---	---	---	---

4.5

C	D	B	A	C
---	---	---	---	---

5.0

C	D	A	B	C
---	---	---	---	---

5.5

D	C	A	B	D
---	---	---	---	---

4.0

C	A	D	B	C
---	---	---	---	---

4.5

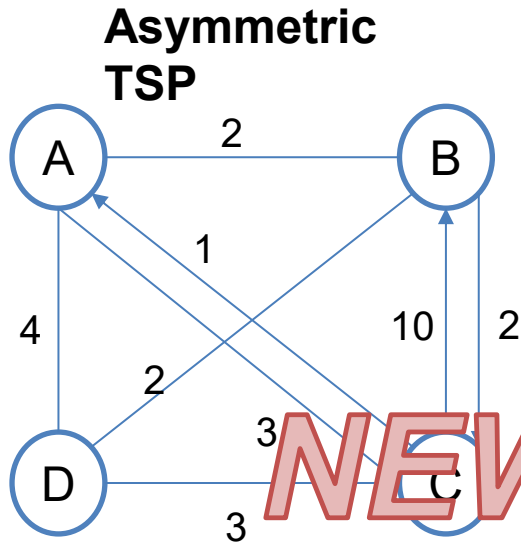
C	D	B	A	C
---	---	---	---	---

5.0

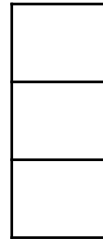
B	D	A	C	B
---	---	---	---	---

9.5

TABU SEARCH



Tabu List
(tenure = 3)



Solution Trajectory

A	B	C	D	A
---	---	---	---	---

11

NEW DATA!

solution = {tour}

move = {swap consecutive pair}

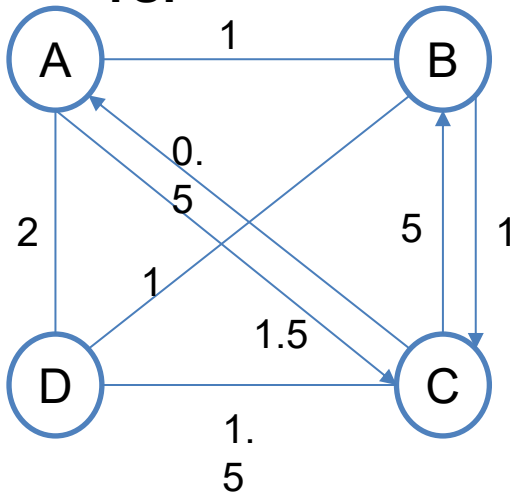
attributes = {moves}

*What could happen if
tenure were 2 instead of
3?*

*What could happen if
tenure were 4 instead of
3?*

TABU SEARCH

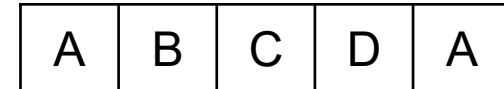
Asymmetric
TSP



Tabu List
(tenure = 2)



Solution Trajectory

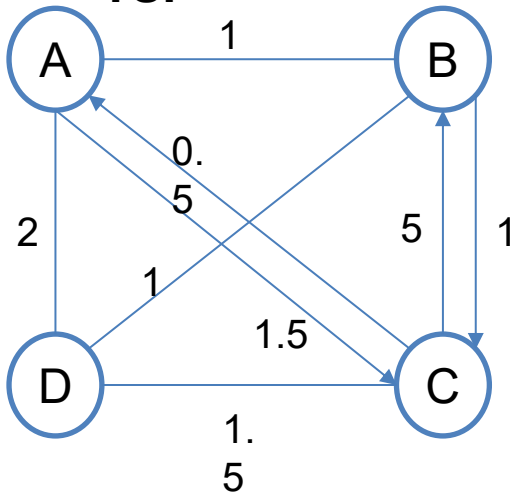


5.5

*What could happen if
tenure were 2 instead of 3?*

TABU SEARCH

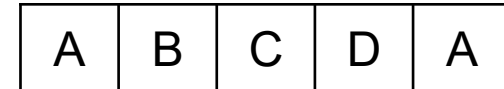
**Asymmetric
TSP**



**Tabu List
(tenure = 4)**



Solution Trajectory



5.5

*What could happen if tenure
were 4 instead of 3?*

TABU SEARCH

- **Decreasing tenure** provides for intensification, though encourages cycling
- **Increasing tenure** provides for diversification by encouraging uphill moves
- How would you design a good TS algorithm?

TABU SEARCH

- Forbidding “attributes” may be overly restrictive
 - aspiration conditions are introduced as “wild-cards” for certain solutions, e.g., new incumbents are always accepted, even if they possess taboo attributes

```
s ← GenerateInitialSolution()
InitializeTabuLists(TL1, ..., TLr)
k ← 0
while termination conditions not met do
    AllowedSet(s, k) ← {s' ∈ N(s) | s does not violate a tabu condition,
                                     or it satisfies at least one aspiration condition}
    s ← ChooseBestOf(AllowedSet(s, k))
    UpdateTabuListsAndAspirationConditions()
    k ← k + 1
endwhile
```

Fig. 4. Algorithm: Tabu Search (TS).

Ant Based Algorithms

Presentation based on Dorigo
et al. 1996: *Ant System:
Optimization by a Colony of
Cooperating Agents.*



Swarm Intelligence

- Collective system capable of accomplishing difficult tasks in dynamic and varied environments without any external guidance or control and with no central coordination
- Each agent is simple, but they can be numerous and together achieve difficult tasks.
- Achieving a collective performance which could not normally be achieved by an individual acting alone
- Constituting a natural model particularly suited to distributed problem solving









Some interesting video links to watch

- Many search algorithms have been inspired by nature or real physical processes. Look at the following videos to get a feel for the physical processes. It will help you understand the algorithms better.
- Cross-over:
<https://www.youtube.com/watch?v=4YfDq8rPDdM>
- Mutations: Mutations are normally considered harmful, but organisms also change for better because of mutation and speciation happens due to mutation as well: <https://www.youtube.com/watch?v=efstlgoynlk>

REAL ANT BEHAVIOR



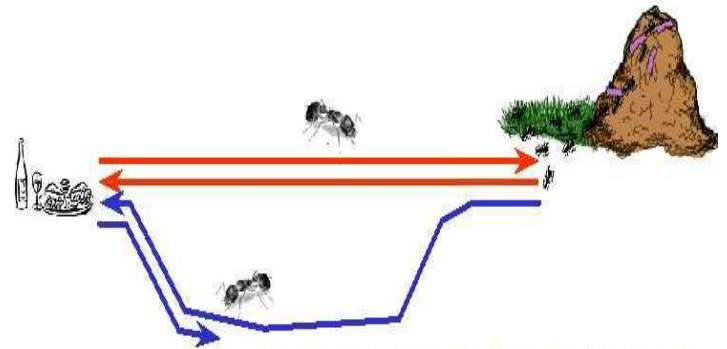
University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

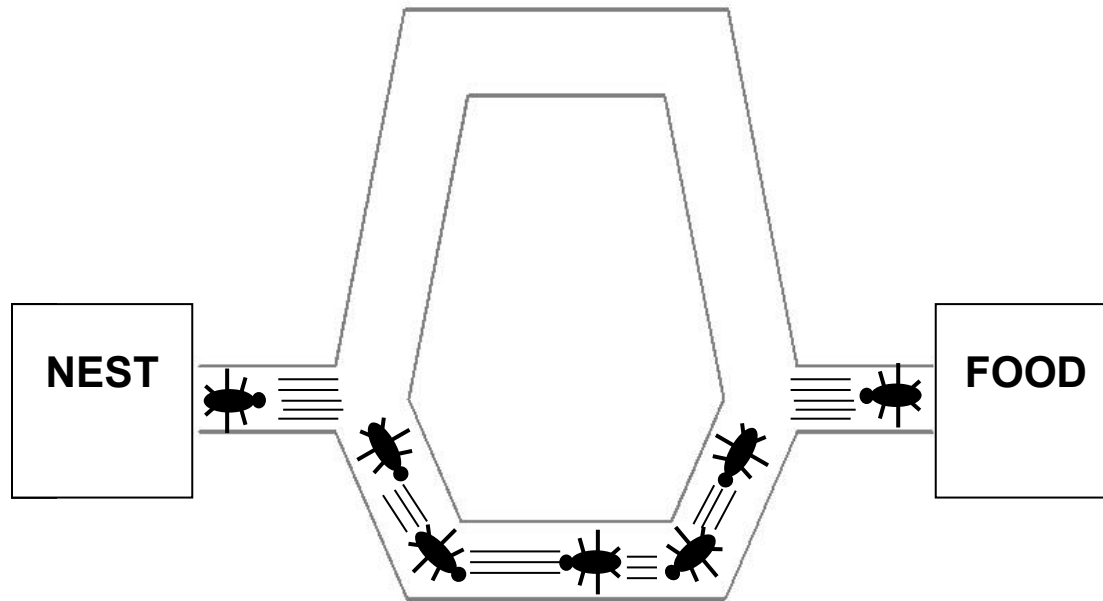
REAL ANT BEHAVIOR

- ❑ Natural behavior of ants have inspired scientists to mimic insect operational methods to solve real-life complex problems
- ❑ By observing ant behavior, scientists have begun to understand their means of communication
- ❑ Ant-based behavioral patterns to address combinatorial problems - first proposed by Dorigo
- ❑ Ants secrete *pheromone* while traveling from the nest to food, and vice versa in order to communicate with one another to find the shortest path



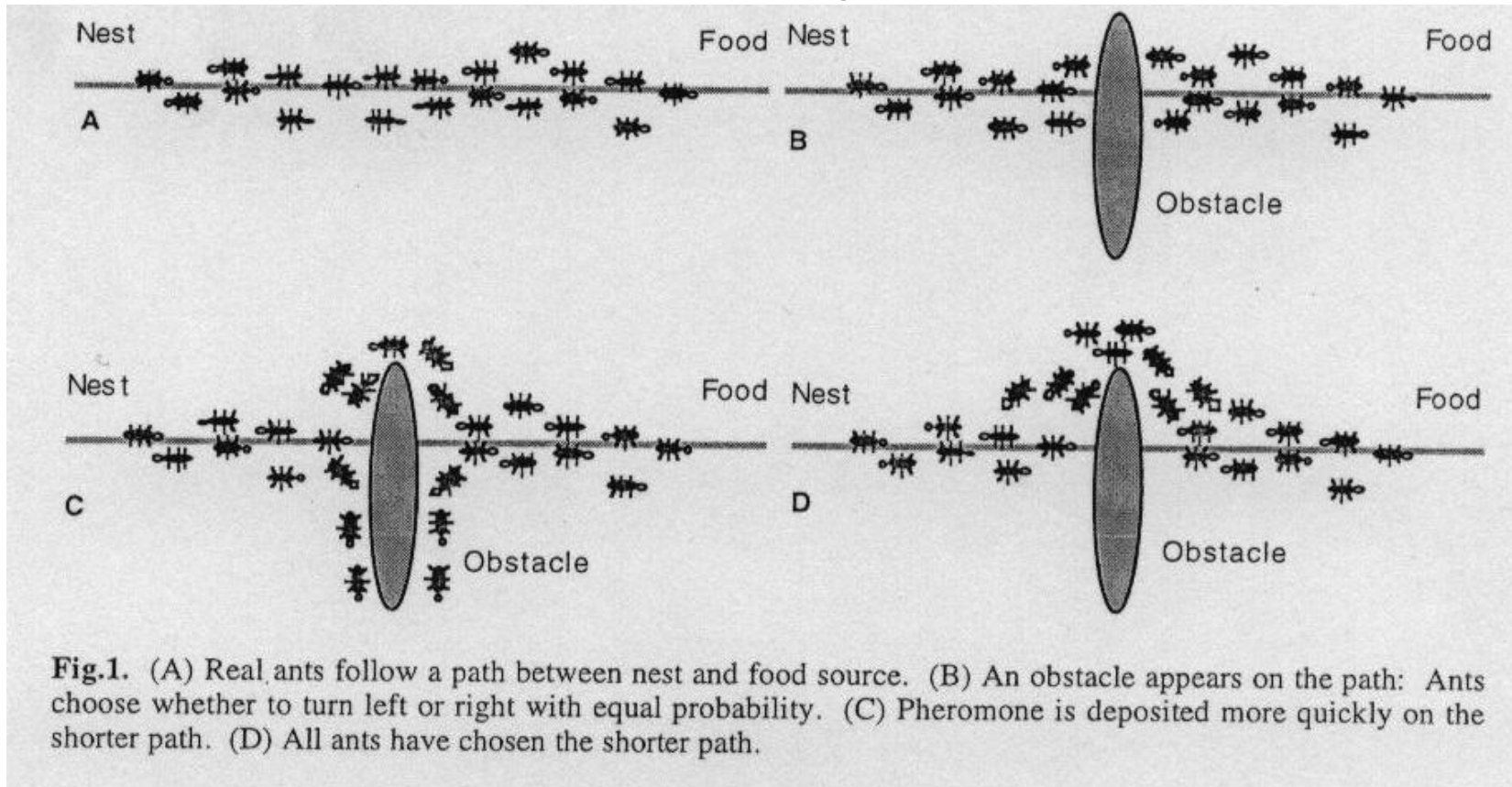
EXPERIMENTAL STUDY OF ANTS

The more ants follow a trail, the more attractive that trail becomes for being followed



ANT Behavior

The more ants follow a trail, the more attractive that trail becomes for being followed



ROUTE SELECTION

- ❑ Initially, ants are forced to decide whether they should go left or right, and the choice that is made is a random decision
- ❑ Pheromone accumulation is faster on the shorter path
- ❑ The difference in pheromone content between the two paths over time makes the ants choose the shorter path
- ❑ Positive feedback mechanism to arrive at the shortest route while foraging
 - ❑ Stigmergy or stigmergetic model of communication: indirect communication among agents to perform actions
- ❑ Different optimization problems have been explored using a simulation of this real ant behavior

An Simple Made-up Example

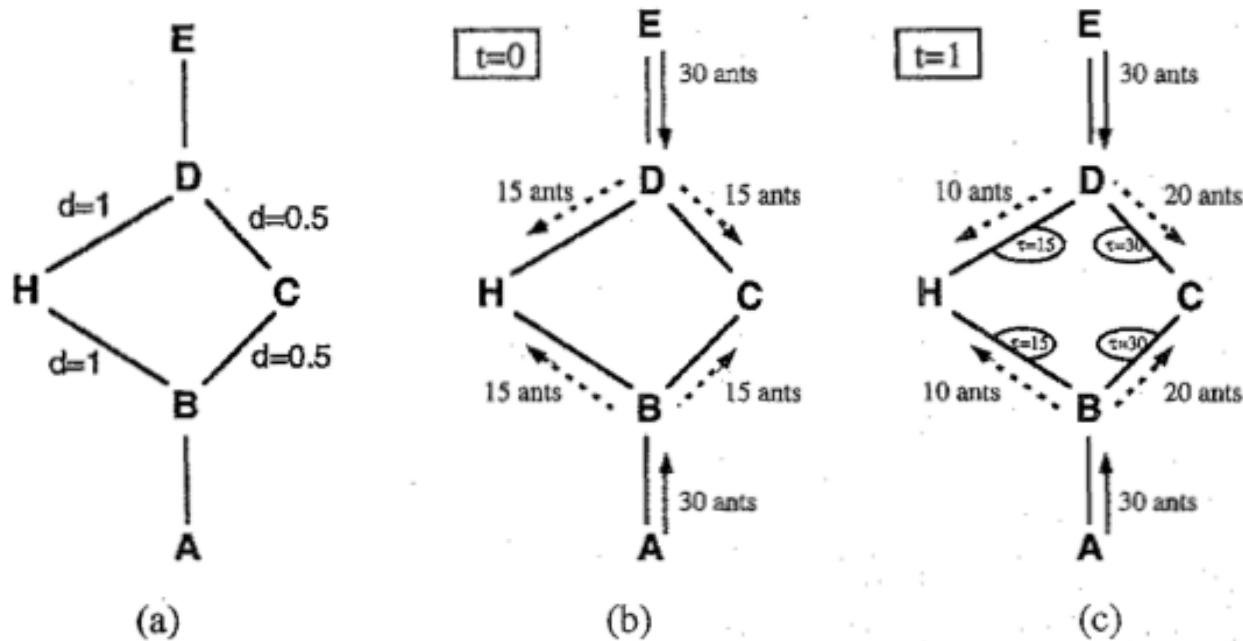


Fig. 2. An example with artificial ants. (a) The initial graph with distances. (b) At time $t = 0$ there is no trail on the graph edges; therefore ants choose whether to turn right or left with equal probability. (c) At time $t = 1$ trail is stronger on shorter edges, which are therefore, in the average, preferred by ants.

The distances are shown as $d=\text{number}$. Assume 30 ants come from A to B at time $t=0$; also 30 ants come from E to D. The ants move at a speed of 1 unit per unit of time.

TRAVELING SALESMAN PROBLEM



University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

PROBLEM DEFINITION

OBJECTIVE

Given a set of n cities, the Traveling Salesman Problem requires a salesman to find the shortest route between the given cities and return to the starting city, while keeping in mind that each city can be visited only once.

The idea is to show that an Ant System can be used to solve the TSP problem first, and then that it can be used to solve other complex problems.

TSP Heuristics

- Variety of heuristics used to solve the TSP
- The TSP is not only theoretically difficult it is also difficult in practical application since the tour breaking constraints get quite numerous
- As a result there have been a variety of methods proposed for the TSP
- Nearest Neighbor is a typical greedy approach

Animations of Ant Algorithms

- <https://www.youtube.com/watch?v=eVKAlufSrHs>
- <https://www.youtube.com/watch?v=8WjUVEamGKA>

THE ANT COLONY OPTIMIZATION METAHEURISTIC

A metaheuristic is a higher-level procedure or heuristic designed to find, generate, or select a lower-level procedure or heuristic (partial search algorithm) that may provide a sufficiently good solution to an optimization problem, especially with incomplete or imperfect information or limited computation capacity (Wikipedia).

GOAL OF ACO HEURISTIC

- ❑ Artificial ants form a multi-agent system performing the functions as observed in the real ant system
- ❑ Exploit stigmergistic communication

The ACO meta-heuristic relies on the co-operation of a group of artificial ants to obtain a good solution to a discrete optimization problem such as the TSP

- ❑ The resulting shortest route mapping determined by the agents can be applied to the optimization problem

ACO CHARACTERISTICS

- ❑ Exploit a positive feedback mechanism
- ❑ Demonstrate a distributed computational architecture
- ❑ Exploit a global data structure that changes dynamically as each ant transverses the route
- ❑ Involves probabilistic transitions among states or rather between nodes

Lecture 9: Search Optimization- Part III

Ant Colony Optimization & differential Evaluation

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

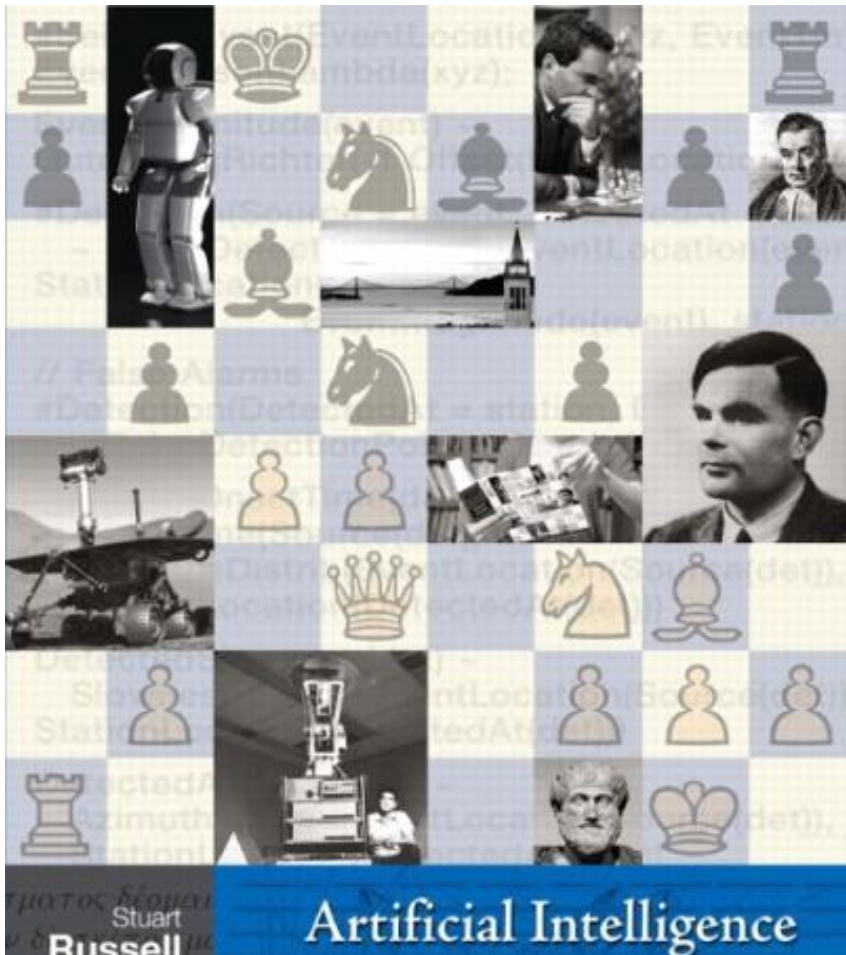
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

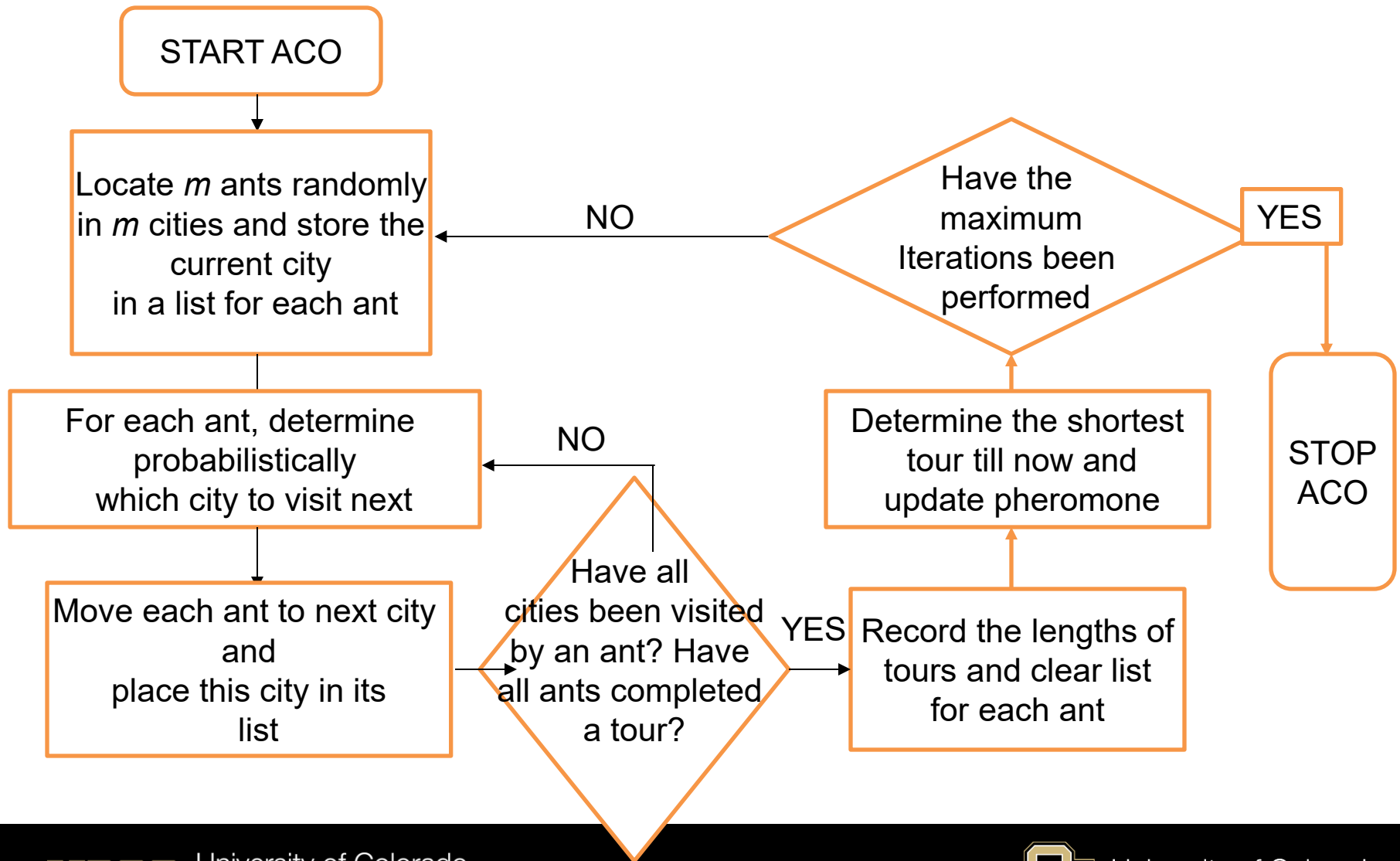
Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Disclosure

- We are using materials from Prof. Panagiotis Repoussis's lecture on metaheuristic algorithms in Stevens Institute of Technology
- We are also using Prof. Jugal Kalita's materials and lecture slides on Ant Colony Optimization

FLOWCHART OF ACO



KEY PARAMETERS

- ❑ Trail intensity is given by value of τ_{ij} which indicates the intensity of the pheromone on the trail segment, (ij) . This variable will keep track of how attractive a trail (an edge of the graph, or an entire tour) is
- ❑ Trail visibility is $\eta_{ij} = 1/d_{ij}$. An ant can see only so far. It's almost blind. It can see closer cities better.
- ❑ The importance of trail intensity and trail visibility are parameterized.
- ❑ The importance of the intensity in the probabilistic transition is α
- ❑ The importance of the visibility of the trail segment is β
- ❑ The pheromone deposited evaporates in time. The trail persistence or evaporation rate is given as ρ
- ❑ Q is a constant and the amount of pheromone laid on a trail segment by an ant. I.e., how much pheromone does ant place on its tour.

PROBABILISTIC CITY SELECTION

- ❑ Helps determine the city to visit next while the ant is in a tour
- ❑ Determined by variables such as the pheromone content on an edge (i,j) at time instant t , heuristic function of the desirability of adding edge, and their control parameters (probability of going from node l to node j at time t for the k_{th} ant) where J_k is the tour the k th ant found.

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in J_k(i)} [\tau_{il}(t)]^\alpha [\eta_{il}]^\beta} & \text{if } j \in J_k(i) \\ 0 & \text{if } j \notin J_k(i) \end{cases}$$

PHEROMONE UPDATING

- Let the length of the completed tour for the k-th Ant be L_k . The quantity of pheromone per unit of length of the trail that this kth ant adds to each edge belonging to the tour is given by

$$\Delta\tau_{ij}^k(t) = \begin{cases} \frac{Q}{L_k} & \text{where edge } (i, j) \in T_k(t) \\ 0 & \text{if edge } (i, j) \notin T_k(t) \end{cases}$$

- The pheromone decay, per unit length, in each edge of a tour is given by

$$\tau_{ij}(t+1) = (1 - \rho)\tau_{ij}(t) + \Delta\tau_{ij}(t)$$

Summary of the ALGORITHM

- ❑ m ants are placed on m nodes randomly
- ❑ Each ant moves from node to node (under constraints that it must build a tour) based on probabilities computed using
 - ❑ Edge attractiveness (how much pheromone)
 - ❑ Edge length (shorter edges are more attractive; greedy approach)
- ❑ Each of the m ants completes a tour
- ❑ The shortest of m tours is chosen.
- ❑ New pheromone is deposited on edges of this most successful tour; some pheromone evaporates.
- ❑ This process is repeated a large number of times

Animations of Ant Algorithms

- <https://www.youtube.com/watch?v=eVKAlufSrHs>
- <https://www.youtube.com/watch?v=8WjUVEamGKA>

Consider the Case Studies in Papers



University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Observations

- ❑ A good number of ants is equal to the number of cities
- ❑ The initial distribution of the ants among the cities does not really matter
- ❑ An elitist strategy in which the segments on the best solution(s) is(are) continually reinforced work well so long as there were not too few or too many elitist solutions
- ❑ Large values of α lead to bad solutions.
- ❑ Very small values of α also lead to bad solutions
- ❑ Dorigo found that values of α and β in the range 1 to 5 seem to work. Problem specific.
- ❑ Dorigo also found that small values of ρ , say 0.5, and values of Q in the range of a few hundred, say $Q=100$, work.
- ❑ There are papers on setting these parameters,

APPLICATIONS (from Wikipedia)

Scheduling problem

- Job-shop scheduling problem (JSP)

- Open-shop scheduling problem (OSP)

- Permutation flow shop problem (PFSP)

 - Single machine total tardiness problem (SMTTP)

- Single machine total weighted tardiness problem (SMTWTP)

- Resource-constrained project scheduling problem (RCPSP)

- Group-shop scheduling problem (GSP)

- Single-machine total tardiness problem with sequence dependent setup times (SMTTPDST)

- Multistage Flowshop Scheduling Problem (MFSP) with sequence dependent setup/changeover times

APPLICATIONS (from Wikipedia)

Vehicle routing problem

- Capacitated vehicle routing problem (CVRP)

- Multi-depot vehicle routing problem (MDVRP)

- Period vehicle routing problem (PVRP)

- Split delivery vehicle routing problem (SDVRP)

- Stochastic vehicle routing problem (SVRP)

- Vehicle routing problem with pick-up and delivery

- (VRPPD)

- Vehicle routing problem with time windows (VRPTW)

- Time Dependent Vehicle Routing Problem with Time Windows (TDVRPTW)

- Vehicle Routing Problem with Time Windows and Multiple Service Workers (VRPTWMS)

APPLICATIONS (from Wikipedia)

Assignment problem

- Quadratic assignment problem (QAP)

- Generalized assignment problem (GAP)

- Frequency assignment problem (FAP)

- Redundancy allocation problem (RAP)

Set problem[edit]

- Set cover problem (SCP)

- Partition problem (SPP)

- Weight constrained graph tree partition problem (WCGTPP)

- Arc-weighted 1-cardinality tree problem (AW1CTP)

- Multiple knapsack problem (MKP)

- Maximum independent set problem (MIS)

APPLICATIONS (from Wikipedia)

Others

Classification

Connection-oriented network routing

Connectionless network routing

Data mining

Discounted cash flows in project scheduling

Distributed Information Retrieval

Grid Workflow Scheduling Problem

Image processing

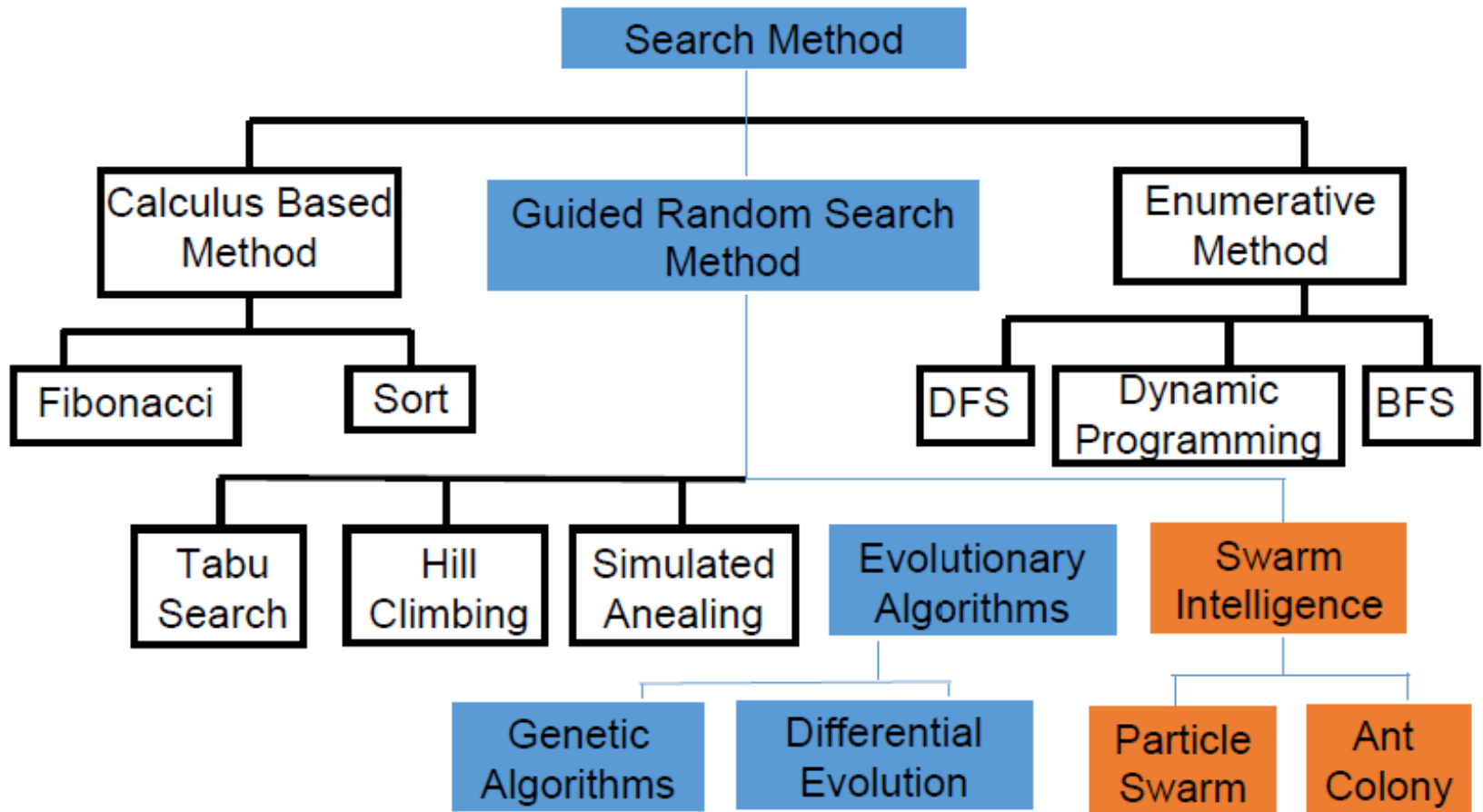
Intelligent testing system

System identification

Protein Folding

Power Electronic Circuit Design

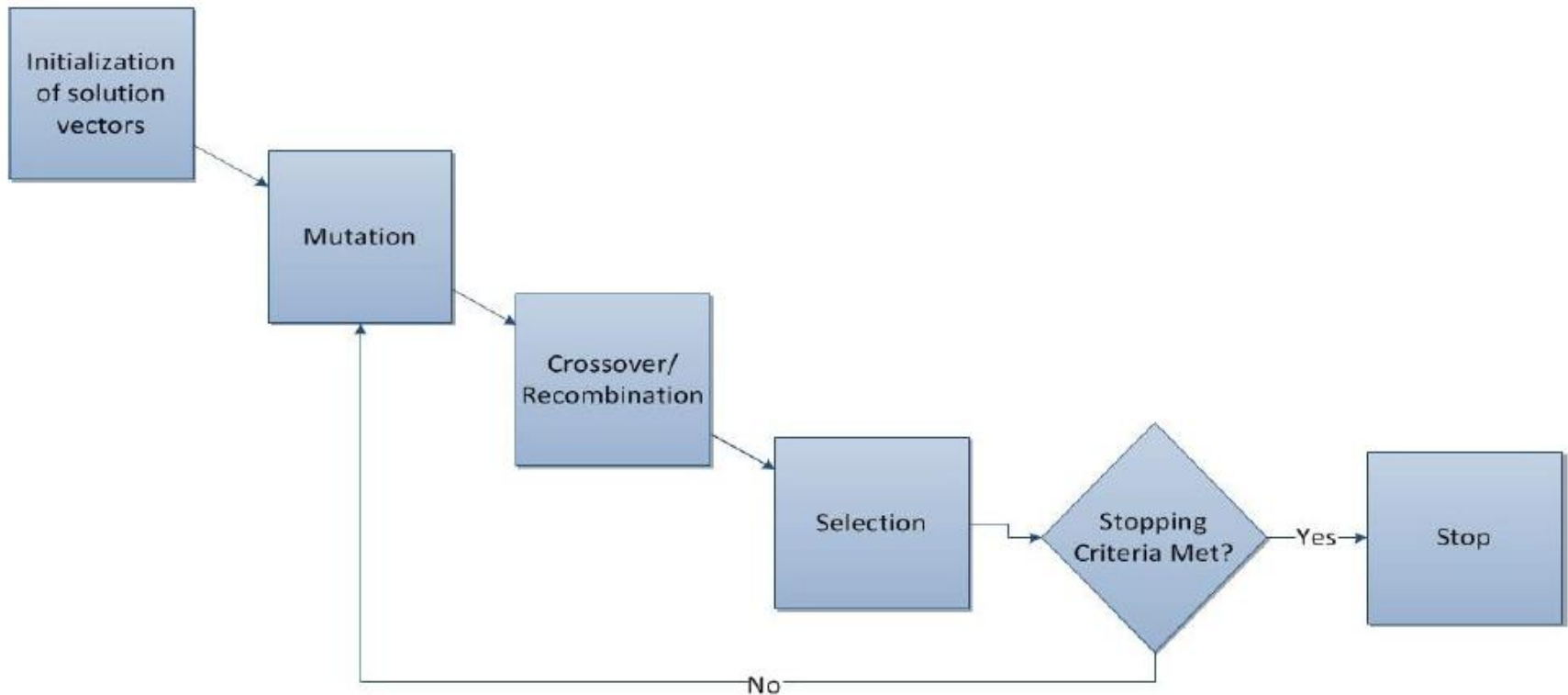
Classes of Search Methods



Differential Evolution

- Real Parameter Optimizer
- Utilizes a pre-specified number of random solutions
- Continuously improves them through a series of mutations and recombinations
- Usually, the number of pre-specified solutions i.e. the population size
- does not change during the lifetime of the algorithm

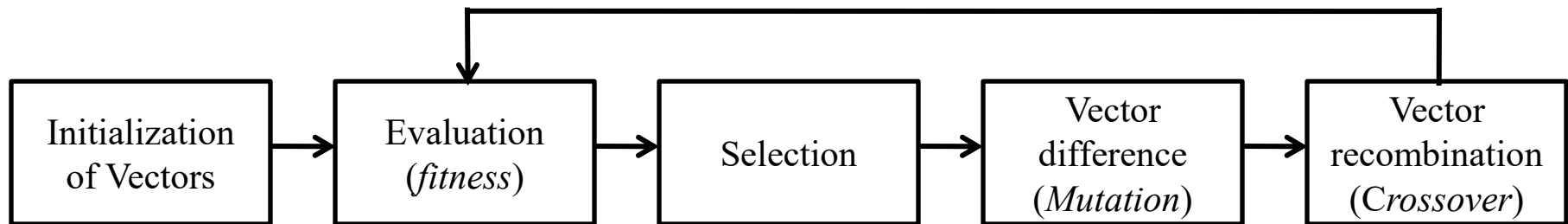
Differential Evolution



Heuristic Methods

Differential Evolution (DE)

- Population based algorithm presented by Storn and Price (1995)
- DeVised to Continuous Search Spaces
- *DE/rand/1/bin*



Important Nomenclature

- D = Dimensionality of the problem
- NP = No. of solution vectors in a population
- G = No. of generations the population has to go through
- F = Scaling Factor
- Cr = Crossover rate

Initialization

- Randomly generate a population of NP , D dimensional real-valued parameter individuals
- Each individual candidate solution is called a vector
- The vectors are changed over generations, denoted by $G = 0, 1, 2, \dots, G_{max}$
- $X_j^i = [x_1^i, x_2^i, x_3^i, \dots, x_D^i]$

Initial Population

- 20 randomly generated initial solutions
- The i^{th} vector solution in the population can be represented as $[x_1^i, x_2^i, x_3^i, \dots, x_D^i]$ where $D=10$ and $NP=20$

$$X_j^i = [x_1^i, x_2^i, x_3^i, \dots, x_D^i]$$

1	2		20
90.34	10.34		10.65
22.43	42.43		42.56
-3.54	-5.54		-5.34
33.23	-3.23		-5.23
54.65	34.65		33.65
22.34	22.34		2.34
-49.31	-49.31		-49.31
23.11	23.11		25.11
-9.33	-19.33		-19.13
-21.22	-31.22		-0.22

Mutation

- Mutation is a change in the gene characteristics of a chromosome.
- When applied to evolutionary computation it means a change in the parameters of the vector through a perturbation with a random element.

Mutation

- A parent vector from the current generation is called **target vector**
- The mutant vector obtained through differential perturbation is called **donor vector**
- The offspring obtained through recombination of **target** and **donor** is called **trial vector**

Mutation

- $V_G^i(\text{Donor}) = X_G^{r_1} + F \times (X_G^{r_2} - X_G^{r_3})$
- F is called the scaling factor and generally lies between 0.5 to 0.9
- The indices r_1 , r_2 and r_3 are mutually exclusive integers randomly chosen from the range $[1, NP]$.

Mutation

$$V_G^i(\text{Donor}) = \begin{matrix} \text{donor} \\ \begin{matrix} 0.65 \\ 2.56 \\ -15.34 \\ -52.23 \\ 3.65 \\ 12.34 \\ -9.31 \\ 2.11 \\ -9.13 \\ -0.22 \end{matrix} \end{matrix} = \begin{matrix} 2 \\ \begin{matrix} 10.34 \\ 42.43 \\ -5.54 \\ -3.23 \\ 34.65 \\ 22.34 \\ -49.31 \\ 23.11 \\ -19.33 \\ -31.22 \end{matrix} \end{matrix} + F \times \left[\begin{matrix} 1 \\ \begin{matrix} 90.34 \\ 22.43 \\ -3.54 \\ 33.23 \\ 54.65 \\ 22.34 \\ -49.31 \\ 23.11 \\ -9.33 \\ -21.22 \end{matrix} \end{matrix} - \begin{matrix} 20 \\ \begin{matrix} 10.65 \\ 42.56 \\ -5.34 \\ -5.23 \\ 33.65 \\ 2.34 \\ -49.31 \\ 25.11 \\ -19.13 \\ -0.22 \end{matrix} \end{matrix} \right]$$

Crossover

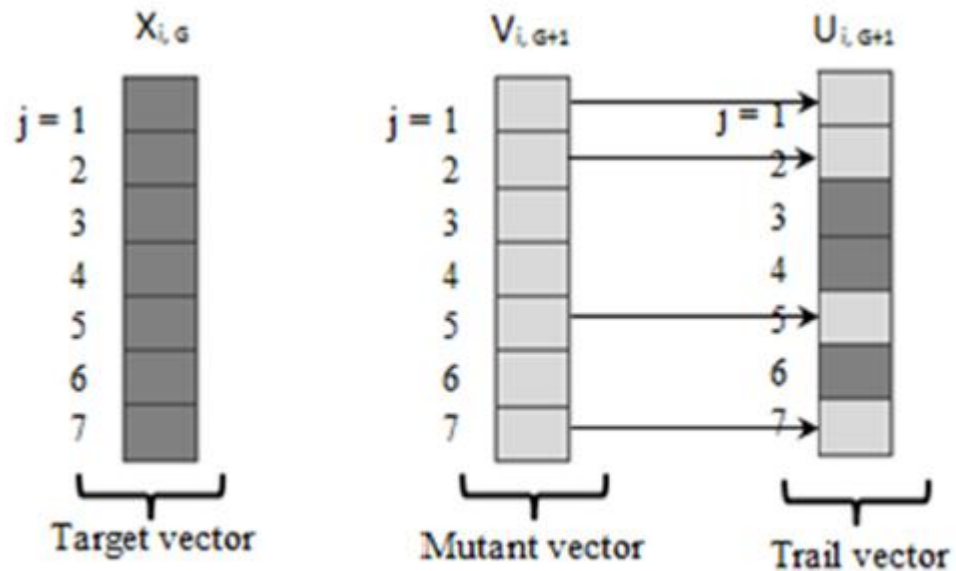


Fig 5. Illustration of Crossover Process of DE with vector dimension (j) of 7. Target vector is current solution with mutant vector is another possible solution. Trail vector is new solution after crossover process between target vector and mutant vector [43].

- Parameters from *target* vector and *donor* vector are selected based on some probability distribution to form a *trial* vector:

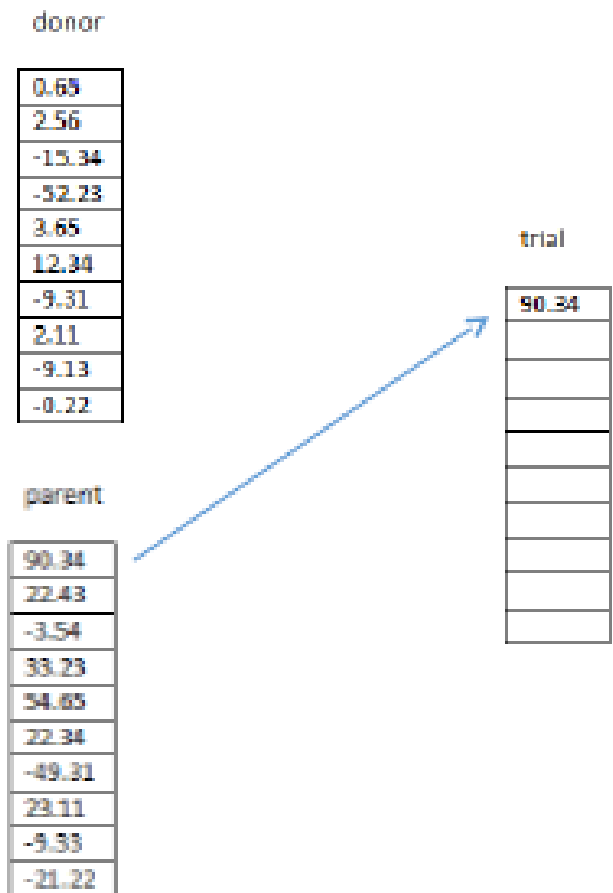
$$u_j^i = \begin{cases} v_j^i & \text{for } j = n, n + 1 \dots n + K - 1 \\ x_j^i & \text{for all other } j \in [1, D] \end{cases}$$

The integer K is drawn from $[1, D]$

n determines the index from where the crossover operation would start.

Create Trial Vector

- Suppose:
 $rand = 0.945$, $Cr = 0.9$
- If $rand < Cr$, pick the parameter from the donor
- Else from the parent
- Do this for all 10 parameters



Selection- Choose between Parent and Child/Trial Vector

- If the fitness of the *trial* vector is better or at least equal to the parent/*target* vector, it moves to the next generation i.e., $G = G+1$ else *target* vector is promoted.
- The process can be described as:

$$\begin{aligned} X_{G+1}^i &= U_G^i && \text{if } f(U_G^i) \leq f(X_G^i) \\ &= X_G^i && \text{if } f(U_G^i) > f(X_G^i) \end{aligned}$$

where $f(X)$ is the objective function.

Control Parameters

- NP, F and Cr are collectively termed as control parameters of DE
- The performance of DE is very sensitive to the values of control parameters
- Every problem/function may respond differently to different sets of control parameters.

Summary of DE

- The exploitation behavior occurs when the difference between two solution vectors in $V_G^i(\text{Donor}) = X_G^{r1} + F \times (X_G^{r2} - X_G^{r3})$ are small while the exploration behavior occurs when the difference between those two are large
- DE is advantageous in terms of enhancing the capacity of local search and keeping the multiplicity of the population while it suffers from slow convergence and being unstable
- Higher population size can benefit DE performance
- Using similar mechanism to LDIW in PSO, scaling factor can be tuned to balance exploration and exploitation behaviors by starting with exploration and gradually turn to do more exploitation.
- Elitism which can avoid the best solution from being destroyed when the next generation is created



University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus